

Pages liminaires (livret v1, 2026-10-09)

Titre. Représentations en grille et en graphe pour l'apprentissage par auto-jeu à Hive : une comparaison pré-enregistrée sous budget de calcul limité (*Grid vs. Graph Representations for Self-Play Learning in Hive: a Pre-Registered Comparison under Limited Compute*)

(Descriptif, sans résultat présumé, selon plan ch. 20.)

Auteur. Mohamed Bechir Kefi

Statut. Rapport de recherche indépendant — non évalué par les pairs, et qui n'est la publication d'aucune institution. Version 1.0-draft, 2026-10-09.

Code et artefacts. Dépôt `hive-graph-selfplay` (étiquette de publication et accès à fixer au moment de la diffusion, G-PUBLIC) ; manifeste des résultats sous `results/` ; chaque figure et chaque table se régénèrent par scripts à partir des enregistrements bruts par partie.

Déclaration d'assistance par IA. Cette étude a été exécutée selon une méthodologie « l'humain comme chercheur principal » (human-as-PI) dans laquelle un assistant de recherche IA (Claude, Anthropic) a implémenté le code, mené les campagnes et rédigé le texte sous un système de portes réservant toutes les décisions scientifiques — gels, budgets, dépenses, publication — à l'auteur humain, qui assume chaque affirmation. Le chapitre 8 documente cette méthodologie de travail en entier ; le document source vit dans le dépôt (`docs/methodology.md`).

Résumé (164 mots dans la version anglaise). Hive est un jeu de stratégie hexagonal sans plateau dont les coups sont des paires (pièce, destination) sur un ensemble de cellules en perpétuel changement — un candidat naturel, en principe, pour des encodages par réseaux de neurones en graphe plutôt que pour les encodages convolutifs en grille standards des systèmes de type AlphaZero. Nous testons cette intuition sous un protocole pré-enregistré, gelé avant toute exécution comparative : un CNN en grille et un réseau relationnel à passage de messages de capacité appariée (+1.5%) partagent un moteur validé côté règles, un décodeur d'actions unique, des réglages d'entraînement identiques, une population gelée de trois adversaires et 250 ouvertures gelées, évalués sous deux lectures budgétaires (à exemples d'entraînement égaux ; à temps mural égal à un seuil pré-enregistré), avec cinq graines indépendantes par bras et la troncature rapportée comme un résultat à part entière. L'hypothèse est rejetée : le bras graphe obtient un score inférieur contre deux des trois adversaires sous les deux lectures, à un coût en temps mural double, son déficit se concentrant dans la conversion des positions gagnées. Les ablations montrent que les relations d'arêtes typées du bras graphe sont porteuses pour la stabilité de l'optimisation, tandis que son agrégation globale (global pooling) est dispensable. Négatif, pré-enregistré et entièrement reproductible à partir des enregistrements publiés.

Mots-clés. Hive ; AlphaZero ; réseaux de neurones en graphe ; apprentissage de représentations ; pré-enregistrement ; résultat négatif

Table des matières. 1 Introduction · 2 Formalisation · 3 Travaux connexes · 4 Méthode (validation du moteur ; lignes de base ; pipeline ; représentations) · 5 Protocole · 6 Résultats (comparaison ; ablations) · 7 Discussion et menaces · 8 Méthodologie de travail (humain-IA sous portes) · 9 Conclusion · Bibliographie · Webographie · Annexe A Corpus de positions · Annexe B Architectures, décodeur,

formats · Annexe C Hyperparamètres, graines, commandes · Annexe D Pointeurs vers le dépôt · Annexe E Tableaux de résultats et figures (générés)

Copie française (D-006) — version 1.0-draft 2026-10-09; les nombres et affirmations sont identiques au master anglais.

Chapitre 1 — Introduction (brouillon du livret, 2026-10-09)

Rédigé en dernier hormis le résumé, selon l'ordre du plan, à partir des résultats finaux.

La question. Hive est un jeu de stratégie hexagonal sans plateau : les pièces définissent la surface de jeu, des piles se forment et se défont, et chaque coup est une paire (pièce, destination) sur un ensemble de cellules qui change à chaque tour. Les encodages en grille — la lingua franca convolutive des systèmes de type AlphaZero — doivent plaquer un cadre sur ce jeu sans cadre. Un encodage en graphe n'a besoin d'aucun cadre. Il est naturel de s'attendre à ce que le graphe l'emporte. Est-ce le cas, à un budget de calcul qu'une seule machine peut se permettre ?

Pourquoi ce n'est pas évident. L'intuition coupe dans les deux sens. Les réseaux en graphe épousent la structure native du jeu et ne portent aucun artefact d'ancrage ; mais l'issue d'une partie de Hive se joue sur des tactiques d'encercllement à courte portée — le régime où la localité convolutive est la plus forte — et les résultats antérieurs divergent : des bras graphe l'ont emporté à Hex sous DQN (Keller et al. 2023) et aux échecs sous auto-jeu (Rigaux & Kashima 2024, à partir d'exécutions uniques), tandis que la seule étude AlphaZero-sur-Hive (de Goede et al. 2022) n'a jamais essayé de graphe. L'hypothèse a été énoncée de façon falsifiable et gelée avant toute exécution comparative, avec tout ce qui pouvait infléchir la réponse : adversaires, ouvertures, budgets, réglages d'évaluation et la règle de rejet elle-même.

Ce que nous avons fait. Nous avons construit les deux encodages derrière un décodeur d'actions partagé unique, sur un moteur validé côté règles, apparié la capacité à +1.5%, et entraîné cinq graines indépendantes par bras sous des réglages d'auto-jeu identiques, en lisant la comparaison de deux façons — à exemples d'entraînement égaux et à temps mural égal — contre une population gelée de trois adversaires sur 250 ouvertures gelées, la troncature étant traitée tout du long comme un résultat de premier ordre.

Ce que nous avons trouvé. L'hypothèse est rejetée. Le bras grille obtient un score supérieur contre deux des trois adversaires sous les deux lectures (plus grande borne supérieure d'intervalle pour un avantage du graphe : +0.028), pour la moitié du coût en temps mural ; le déficit du bras graphe se concentre dans la conversion tactique locale — il gagne du matériel contre l'adversaire aléatoire puis échoue à conclure, tronquant 20–57% de ces parties là où le bras grille n'en tronque presque aucune. Les ablations localisent la machinerie du bras graphe : ses relations d'arêtes typées par direction sont porteuses pour l'optimisation elle-même (les retirer fait diverger l'entraînement pour chaque graine), tandis que son agrégation globale est dispensable.

Contributions. (1) La première comparaison grille-contre-graphe contrôlée, à budget apparié et multi-graines pour Hive, les deux bras sous le même pipeline de type AlphaZero, avec une réponse négative pré-enregistrée — chapitre 6 et [results/comparison/](#). (2) Un harnais de comparaison reproductible pour les jeux sans cadre et à empilement : un moteur de règles validé avec des encodeurs inter-langages épinglés par fichiers dorés (golden-pinned), un décodeur partagé à actions variables, une machinerie d'évaluation consciente de la troncature et une discipline d'artefacts gelés — chapitres 4–5 et le code publié. (3) Une attribution par composant pour le bras graphe (typage des arêtes = entraînabilité ; agrégation globale = effet nul) avec un supplément honnêtement étiqueté — chapitre 6 (H-T3) et [results/ablations/](#).

Chaque contribution est vérifiable depuis le dépôt : chaque nombre de ce livret remonte à une entrée de journal et se régénère par scripts à partir des enregistrements bruts par partie.

Copie française (D-006) — version 1.0-draft 2026-10-09; les nombres et affirmations sont identiques au master anglais.

Chapitre 2 — Formalisation (brouillon du livret, 2026-09-26)

Sources : *protocole gelé v1.0 (D-020)* ; *docs/action-decoder.md (D-015)* ; *docs/representations/* ; *notes d'architecture du moteur*. Aucun nombre ici au-delà de ceux enregistrés.

2.1 Le jeu et la variante étudiée

Hive est un jeu à deux joueurs, à information parfaite et à somme nulle, joué sur un pavage hexagonal non borné : il n'y a pas de plateau — les pièces en jeu définissent la surface de jeu. Nous étudions le **jeu de base uniquement** (reine, araignées, scarabées, sauterelles, fourmis ; 11 pièces par camp), décision D-007. Le moteur implémente les extensions Mosquito/Ladybug/Pillbug ainsi que la restriction d'ouverture de tournoi (pas de reine au premier tour d'un joueur) ; le noyau est exercé par des tests sur les huit types de partie, mais aucun résultat de l'étude ne fait intervenir une extension.

2.2 État, actions, transition, résultat

Un état *s* comprend : les pièces placées avec leurs cellules et leurs niveaux de pile (les scarabées peuvent grimper, produisant des piles), les réserves, le camp au trait, le compte de plis et le marqueur d'étourdissement `last_moved` (pertinent uniquement sous l'extension Pillbug ; conservé pour l'uniformité du noyau — il fait partie de la position hachée). Le moteur expose *s* via le Universal Hive Protocol (UHP) ; la correction de ses règles est établie indépendamment de tout composant d'apprentissage (chapitre 4).

Une **action** est (pièce, cellule de destination), ce qui identifie tout coup de Hive de manière unique — les collisions marche-contre-lancer produisent des états successeurs identiques — plus un *pas* distingué, légal exactement lorsqu'aucun coup n'existe. L'ensemble d'actions légales *A(s)* est produit par le générateur du moteur ; les deux architectures apprises le reçoivent et normalisent leurs politiques exactement sur *A(s)* (le décodeur partagé, chapitre 4). Les transitions sont le `play` du moteur ; la partie se termine quand une reine est entièrement encerclée (victoire pour l'adversaire ; encerclement simultané = nulle), avec nulle additionnelle par triple répétition.

Résultat officiel vs troncature expérimentale. Les parties d'auto-jeu et d'évaluation sont arrêtées à un plafond de 300 plis. Une partie plafonnée n'est **pas** une nulle : c'est une quatrième classe de résultat, *tronquée*, portée à travers le format de données (enregistrements), la perte d'entraînement (les parties tronquées sont exclues de la cible de valeur), le lanceur de matchs, toutes les tables (colonne de taux séparée) et l'analyse. La récompense pour l'apprentissage est le résultat terminal du point de vue du camp au trait : victoire +1, défaite -1, nulle 0 ; la troncature ne contribue à aucune cible de valeur.

2.3 Notation de la recherche et de l'apprentissage

Les deux bras utilisent la même recherche arborescente Monte-Carlo (MCTS) PUCT : à un nœud, un évaluateur renvoie des a priori sur *A(s)* et une valeur scalaire $v \in [-1, 1]$ (point de vue du camp au trait) ; la sélection maximise $Q + c \cdot \text{prior} \cdot \sqrt{N/(1+n)}$ avec $c = 1.4$; les valeurs terminales remontent exactement. L'auto-jeu utilise la randomisation du plafond de simulations (playout-cap randomization : une fraction 0.25 des décisions reçoit 128 simulations et est enregistrée ; le reste en reçoit 32 et ne l'est pas), du bruit de Dirichlet à la racine (ϵ 0.25), un échantillonnage en température sur les 12 premiers plis, et l'abandon sous -0.92 avec une fraction d'audit sans abandon de 10%. **L'évaluation n'applique rien de cette**

machinerie d'exploration ($\epsilon = 0$, argmax déterministe ; imposé par test et épinglé par config, D-019) et exécute 400 simulations par décision.

L'entraînement minimise une entropie croisée de politique contre les distributions de visites MCTS enregistrées, normalisée sur l'ensemble légal pour les deux bras, plus une perte de valeur à 3 classes pondérée (poids 0.6) sur victoire/nulle/défaite, les échantillons tronqués étant exclus. Les budgets sont comptabilisés en quatre dénominations par exécution — temps mural, matériel, états d'entraînement, simulations — et la comparaison est lue sous deux égalisations (exemples égaux, temps mural égal), chapitre 5.

2.4 Vue jeu vs vue encodage

Le même état s est encodé de deux façons (figure fig3-encodings) : la *vue grille* plonge la position dans un cadre fixe 32×32 (déroulé par BFS depuis le plateau torique du moteur, centré sur la boîte englobante) avec 77 plans de caractéristiques ; la *vue graphe* est sans coordonnées — les nœuds sont les cellules occupées plus chaque cellule vide adjacente à la ruche (exactement l'univers de destinations du décodeur), les arêtes portent les six directions hexagonales comme types, les piles apparaissent niveau par niveau comme caractéristiques de nœud, et réserves/camp/pli entrent comme vecteur global. Le chapitre 4 spécifie les deux ; rien d'autre dans le système ne diffère entre les bras.

Copie française (D-006) — version 1.0-draft 2026-10-09; les nombres et affirmations sont identiques au master anglais.

Chapitre 3 — Travaux connexes et positionnement (brouillon du livret, 2026-09-26)

Sources : `docs/reading/` (notes vérifiées ; une ligne par travail dans `docs/reading/matrix.md`) ; décision D-009 (affirmation à portée délimitée). Seules les sources notées et effectivement consultées sont citées ; les métadonnées complètes vivent dans la bibliographie/webographie.

3.1 L'apprentissage par renforcement en auto-jeu à grande échelle — et à petite échelle

AlphaZero (Silver et al. 2017/2018) a fixé le gabarit algorithmique dont hérite cette étude : un réseau politique-valeur guidant un auto-jeu PUCT-MCTS, l'état étant présenté sous forme de plans spatiaux empilés ($8 \times 8 \times 119$ pour les échecs) et la politique elle-même exprimée spatialement — la paramétrisation grille canonique dont descend notre bras grille. Son style de preuve, en revanche, est précisément ce dont une étude à petit calcul doit *s'écarter* : une seule exécution d'entraînement par jeu, des budgets libellés en matériel (5,000 TPU de première génération) plutôt qu'en unités comparables, et une évaluation contre un seul moteur de référence. KataGo (Wu 2020) a montré que le coût du pipeline est compressible d'environ $50\times$ à force égale (moins de 30 V100 pendant 19 jours contre ≈ 74 GPU-années pour ELF OpenGo) et a établi le standard de rapport budgétaire — GPU-jours, parties, échantillons, avec ablations par technique — que suit notre comptabilité ; de ses économies nous en adoptons exactement une, la randomisation du plafond de simulations, parce qu'elle est agnostique à la représentation et peut être appliquée à l'identique aux deux bras, et nous écartons délibérément les caractéristiques d'entrée propres au go et les cibles auxiliaires qui feraient entrer en contrebande de la connaissance du domaine dans un seul encodage. Jones (2021) a entraîné des agents de type AlphaZero à travers plusieurs tailles de plateau de Hex pour un total de ≈ 500 GPU-heures et a trouvé des frontières calcul-performance lisses — la caution méthodologique la plus proche pour tirer des conclusions à notre échelle — et a apporté un avertissement que nous encodons dans le protocole : le calcul à l'entraînement et le calcul au test s'arbitrent l'un contre l'autre, donc les nombres de visites en évaluation doivent être épinglés, et non laissés flottants. Agarwal et al. (2021) fournissent le cadre statistique : dans les régimes à peu d'exécutions, les estimations ponctuelles sur exécutions uniques s'inversent fréquemment sous une analyse par intervalles rigoureuse ; leurs prescriptions — agréger d'abord par exécution, rééchantillonner l'exécution, rapporter des intervalles, ne jamais regrouper les parties comme observations indépendantes — sont mises en œuvre ici avec, tout du long, la graine comme unité de rééchantillonnage.

3.2 Représentations en graphe et espaces d'actions variables

GraphSAGE (Hamilton et al. 2017) fonde la famille inductive à passage de messages dont relève notre bras graphe ; GraphSAGE vanille est dépourvu de sémantique d'arêtes, que nos relations typées par direction ajoutent. Les Pointer Networks (Vinyals et al. 2015) justifient le scoring d'un ensemble variable de candidats — notre décodeur partagé score exactement les paires légales (pièce, destination), les cellules de destination vides étant des objets scorables à part entière. Les réseaux MDP-homomorphes (van der Pol et al. 2020) ne motivent que la question de symétrie *si le temps le permet* ; conformément au protocole, aucune invariance n'est présumée d'un GNN, et aucune n'a été revendiquée.

3.3 Hive et comparaisons grille-contre-graphe

La littérature savante sur l’IA pour Hive est mince. Kampert et al. (2021) ont construit des agents heuristiques minimax/MCTS sur le moteur BeeKeeper, documenté le facteur de branchement ≈ 60 de Hive, et constaté qu’une caractéristique intuitivement centrale (les tuiles autour de la reine) porte étonnamment peu de signal d’évaluation une fois réglée — un avertissement précoce que la structure de valeur de Hive n’est pas là où l’intuition la place ; leurs agents, comme tout agent savant pour Hive avant les nôtres, sont restés en deçà du jeu humain fort. AZ-Hive (de Goede et al. 2022) est le travail antérieur le plus proche et la motivation directe de l’étude : AlphaZero sur Hive à travers un espace de conception 5×2 d’encodages de plateau et d’action — tous des tableaux denses en treillis hexagonal alimentant un CNN, aucune option graphe nulle part dans l’espace (confirmé sur le texte intégral) — avec ce résultat saisissant qu’après 24 h d’entraînement leur meilleur moteur perdait encore contre un MCTS nu et un minimax (BayesElo 1063 contre 1181/1355), tandis que le choix d’encodage changeait mesurablement la vitesse d’apprentissage initiale. C’est précisément la prémisse de RQ-H1 : à Hive, l’encodage est porteur. Polygames (Cazenave et al. 2020) réalise l’invariance à la taille du plateau — corps entièrement convolutifs avec agrégation globale — mais au sein du paradigme grille et sans prise en charge de Hive : il met à l’échelle des plateaux à topologie fixe, ce qu’un jeu sans plateau, à empilement, n’offre pas. Les précédents directs grille-contre-graphe sont au nombre de deux. Keller et al. (2023) ont mené une comparaison CNN-contre-GNN à paramètres appariés sur Hex ($\approx 487K$ contre $\approx 481K$ paramètres, ≈ 110 A100-heures par modèle) et trouvé l’asymétrie que notre résultat prolonge : le GNN dominait les tests de dépendances à longue portée et transférait d’une taille de plateau à l’autre, tandis que le CNN restait plus affûté sur les motifs locaux — mais leur comparaison contrôlée s’est déroulée sous RainbowDQN, leur bras CNN n’a jamais été entraîné sous auto-jeu MCTS, leur graphe est une réduction propre à Hex au jeu de Shannon (les cellules jouées sont contractées ; les actions sont en bijection avec les nœuds), et ils affirment eux-mêmes que la construction ne se généralise pas à d’autres jeux. Rigaux & Kashima (2024, NeurIPS) rapportent le signe opposé pour les échecs : un réseau d’attention sur graphe à caractéristiques d’arêtes (GATEAU) avec une lecture de politique fondée sur les arêtes apprend mieux que les références CNN dans une boucle de type AlphaZero et transfère d’une taille de plateau à l’autre — mais à partir d’une seule exécution d’entraînement par modèle (les intervalles ne couvrent que l’estimation Elo, pas la variance entre exécutions), avec un appariement de capacité lâche (1.0M contre 2.2M paramètres) et un décodeur qui diffère entre les bras, confondant la représentation avec la paramétrisation de l’action ; notre conception élimine exactement ces trois facteurs de confusion. Ben-Assayag & El-Yaniv (2021) ont entraîné un AlphaZero fondé sur GIN sur des graphes de treillis d’Othello pour porter un entraînement sur petit plateau vers des plateaux plus grands — une affirmation de transfert sous budgets délibérément asymétriques, non une comparaison de représentations à budget égal, quoique, de façon notable, avec la meilleure hygiène de réplication des trois (cinq exécutions avec erreurs types). Un projet amateur non publié (hiveGo ; webographie) entraîne un petit GNN sur Hive avec une boucle de type AlphaZero et ne rapporte qu’une évaluation anecdotique — aucune comparaison contrôlée d’aucune sorte.

3.4 Positionnement

Aucun travail antérieur n’exécute de comparaison grille-contre-graphe contrôlée et à budget apparié pour Hive avec les deux bras sous le même pipeline d’auto-jeu ; c’est la contribution à portée délimitée de cette étude (D-009) — délibérément *pas* « la première comparaison grille-contre-graphe dans un jeu de plateau », ce que Keller et al. et Rigaux & Kashima excluent. La portée se délimite comme suit : une variante (Hive de base), un réseau grille et un GNN relationnel simple à capacité appariée, une machine, adversaires/ouvertures/protocole gelés, deux lectures budgétaires, cinq graines par bras. À l’intérieur de ce périmètre, la comparaison est tranchée (chapitre 6) ; en dehors, rien n’est affirmé. Le contraste avec le résultat positif de Rigaux & Kashima aux échecs et avec l’asymétrie longue-portée-contre-locale de Keller et al. est repris dans la discussion.

Copie française (D-006) — version 1.0-draft 2026-10-09; les nombres et affirmations sont identiques au master anglais.

Méthode — Correction du moteur (brouillon de section du rapport)

Rédigé le 2026-09-09 (invariant 14 : écrit au fil de la phase). Master de travail en anglais ; copie française au jalon du rapport (D-006). Chaque nombre est traçable à une entrée de journal (invariant 4) ; les affirmations sont enregistrées dans `claims.md`.

Pourquoi la validation précède tout

Une étude par auto-jeu s'entraîne sur des positions que le moteur génère et évalue lui-même ; un défaut de règles ne se contente donc pas d'ajouter du bruit — il enseigne aux deux bras un jeu systématiquement faux et invalide silencieusement la comparaison (« faux apprentissage », plan ch. 4). La règle d'arrêt est absolue : un moteur trouvé incorrect suspend tout le travail côté entraînement (invariant 11). Nous validons donc selon quatre lignes de preuve indépendantes avant tout travail sur les adversaires de base ou l'entraînement, et nous rapportons chaque ligne avec sa portée exacte.

Quatre lignes de preuve

1. Perft contre les tables publiées. Les comptes de nœuds pour les 8 types de partie correspondent aux tables perft publiées de Mzinga : profondeur ≤ 5 dans la suite de tests standard, profondeur 6 dans une exécution dédiée (8.3 s, tous les types), profondeur 7 dans le script nocturne. Le perft est exhaustif à sa profondeur : toute divergence de la génération de coups — un coup manquant, un coup en trop, une règle d'empilement erronée — décale un compte. (Journal H2-2026-09-09-suite-rerun-01.)

2. Accord avec les moteurs de référence. Le moteur passe le harnais de conformité UHP de nokamute (21/21) et, de manière plus exigeante, le fuzzing différentiel joue des parties aléatoires à graine fixée en vérifiant à chaque demi-coup l'égalité *ensembliste* des coups légaux contre deux moteurs de référence indépendants — MzingaEngine v0.16.0 (l'implémentation de référence UHP) et nokamute 1.0.3 : 27,829 positions à la graine fixe de la session, 200/100 parties par type chaque nuit. Que deux moteurs aux bases de code indépendantes s'accordent sur chaque ensemble de coups légaux borne la probabilité d'une mauvaise lecture partagée des règles. (Journal H2-2026-09-09-suite-rerun-01.)

3. Corpus critique annoté à la main. L'accord avec des moteurs de référence ne peut pas détecter une mauvaise lecture partagée par les moteurs de la communauté ; 30 positions critiques ont donc été annotées *à la main à partir des règles de l'éditeur* (feuille de règles Gen42 et feuille Pillbug, FAQ des World Hive Tournaments ; la règle d'ouverture de tournoi déclarée comme convention) — glissement et portes, empilement du scarabée et porte du scarabée au-dessus du sol, couleur de la pile pour le placement, points d'articulation et anneaux de la règle One-Hive, terminaison victoire/nulle y compris la nulle par encerclement simultané, passe forcée, et l'étourdissement (stun) du Pillbug comme garde au niveau du noyau. Les attendus ont été consignés par commit avant la première exécution du moteur ; le lanceur compare les ensembles complets de coups (pièce, destination) via UHP. Première exécution : 29/30, l'unique désaccord ayant été résolu *contre le corpus* — une ligne de mise en place violait la règle de transit One-Hive que le moteur applique correctement — et 30/30 après la correction ; l'investigation est journalisée dans les deux cas (invariant 2). Limite déclarée : les annotations n'ont pas encore fait l'objet d'une relecture externe par un connaisseur de Hive. (Journal H2-2026-09-09-corpus-run-01.)

4. Sessions d'invariants aléatoires. Des parties aléatoires à graine fixée, sur l'ensemble des 8 types de partie, vérifient après chaque transition : chaque coup généré est accepté par le chemin d'application

(et l’annulation restaure exactement la GameString), la ruche reste connexe, la sérialisation→désérialisation via la GameString UHP reproduit la position, le résultat et l’ensemble exact des coups valides, et la passe est acceptée exactement quand aucun coup n’existe. 10,665,686 coups générés ont été appliqués puis annulés sur 161,546 demi-coups (session approfondie) avec zéro violation ; une session plus petite s’exécute dans la suite standard à chaque build. (Journal H2-2026-09-09-random-invariants-01.)

Portée et lacunes assumées

La couverture de code n’est pas revendiquée comme preuve de correction (plan ch. 4). Les lignes ci-dessus bornent des modes de défaillance différents (exhaustivité à profondeur donnée, accord entre moteurs, fidélité au texte des règles, stabilité des invariants), mais aucune ne prouve la perfection sur parties complètes. Problèmes connus découverts pendant le profilage et reportés à la phase pipeline, consignés comme constats plutôt que corrigés silencieusement : les générateurs d’auto-jeu des travaux antérieurs (*prior-work*) convertissent leur plafond de 300 demi-coups en nulle (l’invariant 7 l’interdit dans tout code de mesure de la nouvelle étude), et le fournisseur d’exécution CoreML de ONNX côté Rust plante actuellement sur la machine d’étude, alors que le même modèle s’exécute sur CoreML via onnxruntime en Python à 2.62 ms/éval. (Journal H2-2026-09-09-throughput-profile-01.)

Vérification inter-langages du chemin de données

L’encodeur de plans Rust du bras grille et le décodeur Python côté entraînement sont maintenus identiques à l’octet près, vérifiés par une contre-vérification sur fichiers de référence (golden files) (240 positions, accord exact) exécutée dans le script nocturne. (Journal H2-2026-09-09-suite-rerun-01.)

Copie française (D-006) — version 1.0-draft 2026-10-09; les nombres et affirmations sont identiques au master anglais.

Méthode — Adversaires de base et population d'évaluation (brouillon de section du rapport)

Rédigé le 2026-09-09 à la clôture de H3 (invariant 14). Master de travail en anglais ; copie française au jalon du rapport (D-006). Les nombres sont traçables au journal H3-2026-09-09-baselines-01 ; les affirmations sont enregistrées dans `claims.md`.

Pourquoi une population gelée

Les deux architectures sont notées par leur résultat moyen contre une **population d'adversaires fixe, gelée avant toute exécution d'entraînement** (protocole §4) : victoire = 1, nulle = 0.5, défaite = 0, les troncatures étant exclues et rapportées comme un taux à part. Une population fixée à l'avance est ce qui rend les scores comparables entre bras, graines et budgets ; toucher un adversaire après coup déplacerait silencieusement l'étalon de mesure, aussi la population est-elle sous discipline de gel — elle a été gelée le 2026-09-09 avec approbation explicite (D-017 : configurations des agents et fichier de poids par sha256, commit du code du moteur) et aucun membre ne peut être ajouté, retiré ou réajusté depuis. Si un nombre de type Elo est cité, il est descriptif et relatif à cette population uniquement.

Les trois agents

Les trois s'exécutent sur le moteur validé (le même noyau de règles que les deux bras de l'étude) via UHP, entièrement contrôlés par graine :

- **B-RND — aléatoire-légal.** Uniforme sur les coups légaux ; déterministe étant donné (graine, historique de la partie). Le plancher et l'ancre du score.
- **B-HEU — heuristique documentée.** Argmax glouton à un demi-coup d'une évaluation construite à la main (sécurité de la reine dominante, mobilité-comme-matériel, petits termes de tempo — chaque caractéristique et chaque poids sont tabulés dans `docs/baselines.md`), plus la quiescence du moteur de recherche sur les coups ciblant la reine adverse. Les poids ont été hérités inchangés du moteur antérieur et sont épinglés par un test automatisé : aucun réglage n'a eu lieu pendant H3 et tout changement ultérieur casse la suite. Cela ferme par construction la voie du réglage-après-observation-des-résultats.
- **B-MCTS — recherche sans réseau.** Recherche arborescente Monte-Carlo (MCTS) PUCT avec a priori uniformes et la même évaluation construite à la main (écrasée par une tanh) comme valeur aux feuilles ; aucun bruit d'exploration à l'évaluation. Budget : 6400 simulations par décision — choisi à partir du coût mesuré (≈ 27 ms/décision en mono-thread sur la machine d'étude), et non des chiffres provisoires du plan.

Vérification de la recherche avant usage

La base MCTS a été vérifiée sur un ensemble tactique annoté à la main, écrit à partir des règles de l'éditeur (même discipline que le corpus de validation du moteur, consigné par commit avant toute exécution) : mat en 1 par marche sur le périmètre et par saut de sauterelle, pour les deux couleurs, et un cas d'évitement d'auto-encerclement — 5/5 à 400, 1600 et 6400 simulations. Les signes de la valeur sous alternance des joueurs sont épinglés par des tests automatisés à trois niveaux : l'évaluation se nie exactement quand le trait change de camp ; l'alpha-bêta note un mat en 1 au-dessus du seuil de mat pour le joueur au trait, quelle que soit la couleur qui joue ; la valeur à la racine du MCTS est fortement positive pour le joueur au trait gagnant, quelle que soit la couleur qui joue.

Caractérisation

100 parties appariées, à couleurs échangées, par confrontation, à partir d'ouvertures aléatoires communes à graine fixée, les troncatures étant rapportées séparément (aucune sur 300 parties) :

Confrontation	V/N/D	Score	Elo (descriptif)
B-HEU vs B-RND	100/0/0	100.0%	$\approx +2400$
B-MCTS vs B-RND	99/1/0	99.5%	+920 [+730, +1200]
B-MCTS vs B-HEU	23/29/48	37.5%	-89 [-150, -32]

Un ordre nous a surpris et est rapporté tel que constaté : la recherche à 6400 simulations se situe *en dessous* de l'heuristique à un demi-coup. Diagnostic (étayé par un sondage à budget $4\times$ atteignant 56.2%) : avec des a priori uniformes, 6400 simulations réparties sur le facteur de branchement de ~ 60 coups de Hive produisent une recherche effectivement peu profonde, tandis que la quiescence de l'agent glouton ciblant la reine est tactiquement tranchante — un écho de la littérature, où la recherche simple et les heuristiques sont fortes à Hive (de Goede et al. 2022 ; Kampert et al. 2021). Aucun réajustement n'a suivi l'observation ; la population couvre délibérément un plancher plus deux adversaires de bande intermédiaire, de styles différents, séparés d'environ 90 Elo.

Limites

Le volume de caractérisation est celui du pilote, 100 parties par confrontation, avec un intervalle en approximation non appariée ; l'ensemble tactique (5 cas) et ses annotations n'ont pas été relus par un lecteur externe connaisseur de Hive — la même limite déclarée que pour le corpus de validation du moteur.

Copie française (D-006) — version 1.0-draft 2026-10-09; les nombres et affirmations sont identiques au master anglais.

Méthode — Pipeline politique-valeur et vérifications pré-entraînement (brouillon de section du rapport)

Rédigé le 2026-09-10 à la clôture de H4 (invariant 14). Master de travail en anglais ; copie française au jalon du rapport (D-006). Les nombres sont traçables aux journaux H4-2026-09-09-coreml-fix-01 et H4-2026-09-10-pilot-01 ; les affirmations sont enregistrées dans `claims.md`.

Le décodeur d'actions partagé

Les deux architectures émettent une politique sur le même espace d'actions — un coup est (emplacement de pièce relatif au camp, cellule de destination) plus la passe — avec un masquage identique de l'ensemble légal, une normalisation sur exactement l'ensemble légal, des cibles de distribution de visites MCTS identiques, et un départage déterministe (`docs/action-decoder.md`, D-015). Le bras grille matérialise cet espace comme un tenseur à 28,673 sorties sur son cadre 32×32 ; le bras graphe (H5) note les paires candidates identiques par position, avec les cellules de destination vides comme nœuds à part entière. Rien dans l'interface d'actions ne diffère entre les bras ; seuls l'encodeur d'état et la fonction de notation diffèrent — c'est le contrôle central des facteurs de confusion de l'étude.

Discipline des données

Les enregistrements d'auto-jeu sont versionnés et auto-descriptifs : chacun porte l'estampille du modèle générateur (génération + hachage du réseau), la position, la cible de distribution de visites, la liste d'indices des coups légaux qui permet l'entraînement masqué, et une issue à quatre valeurs — victoire, nulle, défaite ou **tronquée** (une partie arrêtée au plafond de demi-coups n'est jamais enregistrée comme nulle ; les enregistrements tronqués sont exclus de la perte de valeur). Un manifeste par exécution lie chaque shard à son modèle, et l'agencement des exécutions sépare `selfplay/` (entrées d'entraînement), `checkpoints/` et `eval/` (jamais une entrée d'entraînement), le tout imposé par un script d'audit.

Sept vérifications pré-entraînement

Toutes automatisées, exécutées en une seule commande contre des shards réels avant de faire confiance à tout entraînement : (1) la softmax masquée place une masse exactement nulle sur les actions illégales et somme à un sur les actions légales, sur des positions réelles via une vraie passe avant ; (2) `encode→id→decode` est l'identité sur tous les coups légaux à travers les types de partie (et les cibles stockées sont toujours légales) ; (3) les issues sont correctes pour le joueur au trait, pour les deux couleurs, la troncature restant distincte ; (4) un petit lot fixe sur-apprend jusqu'à un ajustement quasi parfait — `argmax` de politique 15/15, valeur 15/15, KL résiduelle 0.09 contre les cibles douces (le critère est la KL rapportée au plancher d'entropie de la cible : les distributions de visites douces ont une entropie irréductible, donc « perte → 0 » est le mauvais test) ; (5) sauvegarde/reprise reproduit un état du modèle et de l'optimiseur identique au bit près, l'interruption préservant le calendrier de taux d'apprentissage (LR) prévu ; (6) le bruit d'exploration est structurellement impossible à l'évaluation (les valeurs par défaut du chemin d'évaluation portent $\epsilon = 0$, affirmé par test ; réglages épinglés par configuration + décision D-019) ; (7) aucune partie d'évaluation ne peut atteindre l'entraînement, par agencement et par audit.

Pilote à petit budget

Une génération complète à partir d'une initialisation aléatoire à graine fixée (réseau grille de 1.44M paramètres, 300 parties à 128/32 simulations avec randomisation du plafond de déroulés, playout-cap randomization) : 17,237 positions, 56.7% des parties gen-0 tronquées (rapportées, non repliées dans les nulles), entraînement non dégénéré (politique au-dessus du hasard, valeur au-dessus du taux de base, pertes décroissantes), et évaluation indépendante contre la population gelée sous réglages épinglés : **100–0 contre l'aléatoire-légal**, 3.3% contre l'heuristique documentée, 3.3% contre le MCTS à 6400 simulations. Le diagnostic est net et suit l'ordre prescrit (règles, signes, recherche, données — les trois premiers validés indépendamment en H2/H3) : après une génération, l'écart aux adversaires forts est un écart de données et d'itérations, et le levier indiqué est davantage de générations, pas davantage de capacité. Confirmation budgétaire : le coût de génération dans le pire cas (gen-0) est de ≈ 12 s/partie en temps horloge à 4 threads, retombant vers ≈ 3 s/partie à mesure que le jeu s'affine avec un réseau entraîné.

Limites

Une seule graine de bout en bout à l'échelle du pilote ; 30 parties d'évaluation par adversaire ; seul le bras grille existe pour l'instant (le bras graphe arrive en H5 contre le même contrat de décodeur) ; l'itération gen-1+ est exercée structurellement, non exécutée.

Copie française (D-006) — version 1.0-draft 2026-10-09; les nombres et affirmations sont identiques au master anglais.

Méthode — Les deux représentations (brouillon de section du rapport)

Rédigé le 2026-09-10 à la clôture de H5 (invariant 14). Master de travail en anglais ; copie française au jalon du rapport (D-006). Les nombres sont traçables aux journaux H5-2026-09-10-encoders-01 et H5-2026-09-10-graph-wiring-01 ; les affirmations sont enregistrées dans `claims.md`.

Ce qui varie, et ce qui, de manière démontrable, ne varie pas

La variable indépendante de l'étude est la représentation d'état et son corps de réseau — rien d'autre. Les deux bras partagent le moteur, la recherche, les adversaires gelés et les réglages d'évaluation épinglés, les enregistrements d'auto-jeu, les cibles d'entraînement et les conventions d'issue, l'espace d'actions avec sa normalisation sur l'ensemble légal, et la machinerie de la boucle d'entraînement. Chaque élément partagé est imposé plutôt qu'affirmé : le décodeur est un contrat unique avec masquage identique ; les enregistrements sont un format unique ; chaque encodeur est épinglé par un test inter-langages sur fichiers de référence (240 positions pour les plans grille, 160 pour les tenseurs graphe, exact à l'octet près, exécuté chaque nuit) ; et les deux bras s'évaluent via la même recherche arborescente Monte-Carlo (MCTS) en Rust, par des exports ONNX.

Le bras grille

La représentation de base plonge la position dans un cadre fixe 32×32 : dépliée par BFS depuis le tore du moteur, centrée sur la boîte englobante, avec 77 plans de caractéristiques (pièce/propriétaire/type/hauteur conjointement, clouages, étourdissement, régions de placement, et scalaires globaux en plans constants). Une ruche de 28 pièces plus son anneau complet de destinations candidates tient dans le cadre par construction ; cet argument est imposé par une assertion toujours active et des tests extrémaux (lignes droites de 28 pièces : correspondance complète, sans repliement) — aucune pièce ne peut disparaître silencieusement, y compris dans les builds release. Le réseau est un ResNet type KataGo allégé (1.44 M paramètres) avec une tête de politique plate (emplacement de pièce $\times$ cellule du cadre).

Le bras graphe

La représentation graphe est sans coordonnées : les nœuds sont les cellules de l'ensemble candidat — chaque cellule occupée et chaque cellule vide de l'anneau 1, de sorte que chaque destination pouvant être notée est un nœud à part entière ; les arêtes sont typées par les six directions hexagonales ; les caractéristiques de nœud portent la pile niveau par niveau (propriétaire et type d'insecte par niveau), les clouages, l'étourdissement et les régions de placement ; les réserves, le trait, le numéro de demi-coup et le type de partie entrent comme vecteur global. Les pièces ne sont pas des nœuds séparés : le décodeur partagé adresse les coups comme (emplacement de pièce, cellule de destination), de sorte que l'identité de la pièce entre dans la politique par un plongement d'emplacement et par le nœud où la pièce se tient — un choix de conception documenté, pas un accident. Le réseau est un petit réseau relationnel à passage de messages (poids propres à chaque direction, biais d'agrégation globale, agrégation masquée pour la tête de valeur, notation des coups par candidat) de 1.47 M paramètres — une différence de capacité de +1.5%, rapportée.

Un réseau de graphe n'offre gratuitement ni invariance ni équivalence de règles.

L'encodage ne contient aucune coordonnée absolue, mais la fonction apprise n'en est pas pour autant

invariante par translation ou par rotation, et aucune règle de Hive n'y est câblée ; toute affirmation de ce type dans cette étude est mesurée, jamais supposée.

Asymétries de coût mesurées (rapportées, non égalisées)

Sur la machine d'étude, le meilleur chemin d'inférence disponible diffère selon le bras : le réseau grille convolutif s'exécute sur l'accélérateur CoreML à 2.62 ms/évaluation, tandis que le réseau graphe, riche en opérations de collecte (gather), s'exécute le plus vite sur CPU à 3.67 ms (CoreML est plus lent pour lui) — un coût par décision de $\approx 1.4\times$ au détriment du bras graphe. L'entraînement montre le motif inverse selon le périphérique (le CPU favorise le graphe $3.5\times$; le chemin MPS utilisé pour l'entraînement favorise la grille $\approx 2\times$). Ce sont de véritables interactions matérielles des représentations ; les deux lectures budgétaires du protocole (mêmes-exemples et même-temps-horloge) les facturent honnêtement plutôt que de les cacher.

Augmentation par symétries

Exclue de la méthode complète pour les deux bras (D-023) : l'exclusion rend trivialement vraie la règle du protocole de données identiques, évite d'implémenter les symétries hexagonales différemment selon la représentation, et laisse l'augmentation comme une question d'ablation additive propre.

Limites

Les coûts sont mesurés sur une seule machine ; la validation du câblage n'a entraîné le bras graphe qu'à l'échelle d'un test de fumée (son profil d'apprentissage sur les données gen-o a correspondu à celui du bras grille, comme attendu pour des bras appariés sur données partagées — ce n'est pas un résultat de comparaison) ; la comparaison contrôlée elle-même est H6, sous le protocole v1.0.

Copie française (D-006) — version 1.0-draft 2026-10-09; les nombres et affirmations sont identiques au master anglais.

Chapitre 5 — Protocole expérimental (brouillon du livret, 2026-09-26)

Source d'autorité : le protocole gelé v1.0 (D-020, sha256 f340a6b6...aefeb5) et les décisions de gel et d'épingleage D-017, D-019, D-025, D-026, D-027. Ce chapitre les reformule pour le lecteur ; le document gelé prévaut en cas de divergence.

5.1 Discipline de pré-enregistrement

Tout ce qui pouvait biaiser la comparaison a été gelé avant l'existence de toute exécution de comparaison, dans cet ordre : population d'adversaires (D-017), protocole avec sa règle de rejet et ses budgets mesurés (D-020), ouvertures fixes partagées (D-025) ; les réglages d'évaluation ont été épinglés par configuration et par test (D-019). Le seuil à temps mural égal T^* a été calculé selon une règle énoncée dans la matrice pré-enregistrée — la médiane des temps muraux d'exécution complète des trois exécutions du bras grille — à partir des seuls chronomètres de la grille, avant l'assemblage de tout nombre inter-bras. Aucun artefact gelé n'a été modifié à aucun moment ; l'historique du dépôt le documente.

5.2 Matrice, graines, matériel, budgets

2 architectures $\times$ 3 graines d'entraînement (1–3 ; espaces de graines disjoints), 10 générations $\times$ 500 parties d'auto-jeu par exécution à 128 simulations complètes / 32 simulations économiques par décision (choix mesurés, et non les valeurs de substitution du plan), plafond de 300 demi-coups. Une seule machine (Apple M1 Pro, 10 cœurs, 16 Go), quatre threads de travail par exécution, exécutions séquentielles ; chaque bras utilise son meilleur fournisseur d'inférence mesuré (grille : CoreML à 2.62 ms/eval ; graphe : CPU à 3.67 ms/eval) — une interaction matérielle réelle, rapportée et imputée, non neutralisée par égalisation. Chaque exécution journalise le temps mural par génération, les états et les simulations ; les points de contrôle sont conservés à chaque génération.

5.3 Adversaires, ouvertures, appariement

La population d'évaluation est gelée : un adversaire aléatoire légal à graine fixée (B-RND), une heuristique documentée à poids fixes (B-HEU ; poids épinglés par hachage et imposés par test), et un MCTS sans réseau à 6400 simulations (B-MCTS). Le point de contrôle gen-19 de la boucle de démonstration antérieure est exclu (D-016/17). Les ouvertures sont 250 ouvertures de 4 demi-coups pré-générées, uniques et légales (générateur à graine fixée, hachage de contenu enregistré) ; chaque affrontement joue la paire i sur la ligne d'ouverture i avec les couleurs inversées, de sorte que tous les bras, graines et adversaires font face à des calendriers d'ouvertures et de couleurs identiques (vérifié par le test de hachage du calendrier).

5.4 Métriques, intervalles, exclusions

Métrique primaire : le score moyen (victoire 1 / nulle 0.5 / défaite 0) contre la population, par cellule (graine, adversaire), **en excluant les parties tronquées**, dont le taux est toujours rapporté séparément, avec une colonne de sensibilité « troncatures à 0.5 » et des bornes de traitement du plafond (troncatures comptées comme défaites et comme victoires) encadrant tout plafond alternatif. L'unité de rééchantillonnage est la **graine** : les cellules agrègent d'abord les parties ; les intervalles sont des bootstrap percentiles (10,000 rééchantillonnages) sur les trois graines ; le contraste entre bras applique

le bootstrap indépendamment aux ensembles de graines des deux bras. Les parties ne sont jamais agrégées comme i.i.d. ; l'Elo, là où l'outillage l'imprime, est descriptif seulement. La sélection des points de contrôle est mécanique : les points de contrôle de la génération 10 pour la lecture à exemples égaux ; le dernier point de contrôle achevé à $\leq T^*$ pour la lecture à temps mural égal. **Critères d'exclusion** : aucun n'a été nécessaire — aucune exécution n'a échoué, aucune n'a été exclue, et un mauvais score n'est pas un critère d'exclusion selon le protocole.

5.5 Matrice RQ → expérience → résultat

RQ	Expérience	Artefact de résultat
RQ-H1 : le bras graphe bat-il le bras grille à budget comparable ?	La matrice H6 sous les deux lectures contre la population gelée	Tables de scores par graine + contraste entre bras avec intervalles bootstrap sur les graines (results-same-examples / results-same-wallclock / results-arm-difference) ; verdict via la règle de rejet gelée
RQ-H2 (secondaire) : lecture par exemple vs par heure	La même campagne lue aux points de contrôle de la génération 10 vs aux points de contrôle T^*	Courbes score-vs-temps (fig1) ; table score/coût (fig2)
RQ-H3 (secondaire) : que contribuent les composants du graphe ?	Deux ablations à un composant à parité (A1 typage des arêtes ; A2 pooling global, substitution D-028)	Table d'ablation H-T3 (results/ablations/) ; supplément A1' (deux composants, clairement étiqueté) s'il est exécuté

5.6 Reproduction

Chaque table et chaque figure se régénèrent par scripts à partir des enregistrements bruts par partie (`scripts/make_results.py`, `make_figures.py`) ; les réglages de générateur, graines et empreintes de modèle de chaque exécution résident dans des manifestes par exécution ; commandes, configurations et graines sont en annexe. Le scénario de reproduction minimal (annexe) rejoue de manière déterministe une seule partie d'évaluation enregistrée et régénère les tables de résultats à partir des enregistrements livrés.

Copie française (D-006) — version 1.0-draft 2026-10-09; les nombres et affirmations sont identiques au master anglais.

Résultats — La comparaison contrôlée (brouillon de section du rapport)

Rédigé le 2026-09-19 à la clôture de H6 (invariant 14). Master anglais de travail ; copie française au jalon du rapport (D-006). Chaque nombre remonte aux journaux H6-2026-09-19-comparison-01 (analyse à 3 graines) et H6-2026-10-09-5seed-final-01 (analyse finale à 5 graines) et se régénère via `scripts/make_results.py / make_figures.py` sur les enregistrements bruts par partie ; affirmations consignées dans `claims.md`.

La question pré-enregistrée et sa réponse

Le protocole (gelé en v1.0 avant toute exécution de comparaison) demandait : à budget d'entraînement comparable, l'architecture graphe apprend-elle une meilleure politique que l'architecture grille pour Hive en jeu de base ? Il fixait, à l'avance, ce qui rejetterait l'hypothèse : aucun avantage graphe cohérent entre graines sous les deux lectures de budget, avec des intervalles excluant un avantage graphe substantiel.

C'est ce qui s'est produit. H1 est rejetée. Sous la lecture à exemples égaux (les deux bras, 10 générations $\times$ 500 parties) et la lecture à temps mural égal ($T^* = 18.77$ h, calculé par une règle pré-enregistrée avant l'existence de tout nombre inter-bras), le bras graphe a obtenu un score plus faible contre deux des trois adversaires gelés et pas meilleur contre le troisième, sur **cinq graines par bras** (les graines 4–5 ont été ajoutées après l'analyse à trois graines, symétriquement et sous un pré-engagement d'utiliser toutes les graines quelle que soit la direction, D-031 ; l'analyse à trois graines a abouti au même verdict) :

graphe – grille	Exemples égaux	Temps mural égal
vs aléatoire légal	−0.169 [−0.272, −0.062]	−0.161 [−0.278, −0.048]
vs heuristique	−0.064 [−0.111, −0.017]	−0.059 [−0.087, −0.029]
vs MCTS à 6400 sim	−0.009 [−0.055, +0.028]	−0.018 [−0.066, +0.025]

(Moyennes au niveau des graines ; bootstrap à 95% sur cinq graines par bras ; valeurs par graine dans les tables de résultats — aucun rapport de « meilleure graine » nulle part. La plus grande borne supérieure sur les six contrastes est +0.028.)

La lecture à temps mural aggrave le résultat : le coût mesuré du bras graphe était de $2.0\times$ par exécution (36.7 vs 18.0 h de temps mural d'entraînement, moyenne sur les cinq graines), de sorte qu'à heures égales il ne complète que 3–6 des 10 générations (4–5 sur les trois graines d'origine) — un déficit que la lecture par exemple montre déjà et que le temps égal ne fait qu'élargir.

La fig. 5 décompose les scores agrégés en trajectoires par adversaire pour les 10 exécutions de la campagne principale ; la fig. 6 rapporte les métriques d'ajustement d'entraînement (politique et valeur) par génération — les deux bras ajustent leurs données d'auto-jeu tout du long, de sorte que l'écart n'est pas un simple échec d'optimisation.

La troncature, rapportée séparément et mise à l'épreuve

La différence comportementale la plus nette n'est pas un score mais une catégorie d'issue : contre l'aléatoire légal, le bras graphe a tronqué 20–57% de ses parties au plafond de 300 demi-coups sur les

graines originales et 23–44% sur les graines d’extension (grille : 0–1%, avec une graine d’extension à 11%) — gagnant du matériel puis échouant à convertir (fig. 4, F1). Parce que la troncature a été définie dès le départ comme une issue à part entière, cette pathologie est visible au lieu d’être blanchie en nulles ; la fig. 7 trace les taux par exécution contre l’aléatoire légal. La valeur du plafond ne peut pas sauver l’hypothèse : même en comptant chaque partie tronquée comme une victoire du graphe — une borne supérieure pour tout plafond plus grand — le bras graphe reste derrière sur l’adversaire aléatoire sous les deux lectures (-0.072 / -0.058).

Ce que cela montre et ne montre pas

Cela montre : pour un réseau relationnel simple à passage de messages, à capacité appariée (+1.5%), partageant tous les autres composants avec le bras grille — règles, recherche, décodeur, enregistrements, conventions d’entraînement, adversaires gelés, ouvertures gelées, évaluation épinglée — l’encodage en plans de grille a appris davantage par exemple ET par heure à ce petit budget, et la faiblesse du bras graphe se concentre dans la conversion tactique locale, en accord avec l’asymétrie local-vs-longue-portée rapportée pour Hex sous DQN (Keller et al. 2023), ici observée sous auto-jeu apparié de style AlphaZero dans un jeu sans cadre, à empilement.

Cela ne montre pas : quoi que ce soit sur les représentations en graphe à plus grands budgets, sur d’autres architectures de graphe ou sur d’autres jeux ; ni que les bras ne se réordonneraient pas avec davantage de générations (10 relève du régime précoce ; tous les scores contre les lignes de base fortes restent bas). Cinq graines par bras bornent la statistique ; le rejet est cohérent entre graines, et les graines d’extension (ajoutées sous pré-engagement) ont resserré quatre des six intervalles.

Coûts (dans les deux dénominations, selon plan ch. 6)

Fig. 2 : paramètres 1.44M vs 1.47M ; inférence au meilleur fournisseur 2.62 ms (CoreML) vs 3.67 ms (CPU) ; auto-jeu 13.0 vs 26.4 s/partie (temps mural d’entraînement / 5,000 parties) ; score contre la population 0.411 vs 0.331 (exemples égaux), 0.405 vs 0.326 (temps mural égal) — moyennes sur cinq graines.

Provenance

Population gelée le 2026-09-09 (D-017) ; protocole gelé le 2026-09-10 (D-020) ; ouvertures gelées le 2026-09-10 (D-025) — le tout avant toute exécution de comparaison. Campagne approuvée et lancée le 2026-09-10 (D-026) ; T^* calculé le 2026-09-16 à partir des seuls temps muraux de la grille ; aucune retouche d’adversaire, d’ouverture ou de protocole n’est survenue à aucun moment. Résultat négatif conservé et rapporté selon l’invariant 5 et le protocole §1 : le travail n’a pas besoin que H1 soit confirmée pour compter.

Les ablations (H-T3)

Deux ablations à un seul composant contre la méthode graphe complète, trois graines chacune à pleine parité de budget, plus un supplément étiqueté. Retirer les relations d’arêtes typées par direction (A1, adjacence naïve) a détruit d’emblée l’entraînabilité — divergence NaN à la génération 0 pour chaque graine — de sorte que les arêtes typées portent, au minimum, la stabilité d’optimisation du bras. Retirer le biais de pooling global (A2) a produit un résultat nul : des variations de score de -0.001 [-0.170 , $+0.165$], -0.007 [-0.043 , $+0.030$] et -0.013 [-0.043 , $+0.013$] contre les trois adversaires, avec des modes de défaillance inchangés. Le supplément A1’ (arêtes non typées plus écrêtage de gradient — une variante explicitement à deux composants, jamais attribuée au seul typage) s’entraîne de façon stable et obtient des scores dans la bande du bras complet contre les trois adversaires, ce qui suggère que la

contribution mesurable des relations typées à cette échelle se concentre dans la stabilité d'optimisation plutôt que dans la force finale — un énoncé qui hérite du facteur de confusion de l'écrêtage et qui est formulé en conséquence partout où il apparaît.

Copie française (D-006) — version 1.0-draft 2026-10-09; les nombres et affirmations sont identiques au master anglais.

Chapitre 7 — Discussion et menaces à la validité (brouillon du livret, 2026-09-26)

Découpage en quatre volets selon plan ch. 21. Ancré dans le registre des affirmations.

7.1 Lire le résultat

H1 a été rejetée exactement comme le protocole gelé définissait le rejet. Trois observations donnent au résultat sa texture. Premièrement, le déficit est modelé par l'adversaire : le plus grand contre l'aléatoire légal, net contre l'heuristique, absent contre le B-MCTS riche en recherche — cohérent avec un bras graphe qui est le plus faible en *conversion tactique locale* plutôt qu'en jugement positionnel. Deuxièmement, le canal de troncature porte le mécanisme : le bras graphe gagne du matériel contre l'aléatoire puis échoue à conclure (20–57% de parties plafonnées ; fig4-F1), un motif invisible dans les études qui replient les plafonds en nulles. Troisièmement, les ablations affinent la lecture : retirer le typage géométrique des arêtes n'affaiblit pas simplement le bras graphe — aux réglages de parité, cela détruit d'emblée l'entraînabilité (divergence NaN, génération 0, 3/3 graines), de sorte que les relations typées portent au minimum la stabilité d'optimisation du bras entier — tandis que la seconde ablation a trouvé le biais de pooling global entièrement dispensable (variations de score $\approx 0.00 \pm 0.17$, -0.01 ± 0.04 , -0.01 ± 0.03 contre les trois adversaires ; modes de défaillance inchangés), localisant la machinerie distinctive du bras dans les relations directionnelles, non dans le pooling. Cela contraste de manière instructive avec le résultat positif aux échecs de Rigaux & Kashima (2024, NeurIPS) et l'asymétrie de Keller et al. (2023) sur Hex : notre résultat ne les contredit pas — il borne là où leur optimisme se transfère, et la comparaison des standards de preuve importe : le résultat aux échecs repose sur une seule exécution d'entraînement par modèle avec des intervalles couvrant la seule estimation Elo, tandis que le présent rejet est cohérent entre graines sur cinq exécutions par bras sous deux lectures de budget pré-enregistrées. La tactique de Hive est dominée par la géométrie d'encerclement à courte portée, le régime où Keller et al. ont trouvé les CNN plus forts ; aux petits budgets, ce régime décide des parties.

7.2 Validité interne (bogues, comparabilité)

Le moteur est validé indépendamment de l'apprentissage (perft jusqu'à la profondeur 6+ contre des tables publiées, conformité UHP 21/21, accord différentiel sur 27,829 positions avec deux moteurs de référence, un corpus annoté à la main dérivé des règles, des sessions d'invariants à 10.9M transitions). Les deux encodeurs sont épinglés à l'octet près contre des goldens inter-langages exécutés chaque nuit. Les bras partagent le décodeur, les enregistrements, les fonctions de perte, les budgets et la recherche ; la capacité diffère de +1.5% (rapporté). Risques résiduels : l'architecture graphe est UN point de l'espace de conception — un GNN plus fort pourrait se comporter différemment (nous n'affirmons rien au-delà de ce réseau) ; des défauts d'outillage trouvés pendant l'étude (un chemin de perte d'enregistrements d'arène sur les affrontements entièrement tronqués ; un passage NaN silencieux dans l'entraînement d'ablation) ont été attrapés par la discipline de vérification, corrigés, et audités comme ayant laissé les données de campagne intactes — mais ils illustrent que l'erreur de harnais, et non le hasard, est le mode de défaillance dominant à cette échelle.

7.3 Validité de mesure (adversaires, troncature)

La population est constituée de trois adversaires fixes couvrant une force du plancher au niveau intermédiaire ; tous les bras entraînés perdent encore lourdement contre les deux lignes de base fortes,

de sorte que la comparaison vit dans un régime de scores bas où les différences contre B-MCTS sont difficiles à résoudre. Les scores sont relatifs à CETTE population — aucune affirmation de force universelle n’est faite. La troncature est traitée comme une issue à part entière avec des bornes de sensibilité ; le plafond de 300 demi-coups lui-même ne peut pas renverser le verdict (la borne « troncatures comptées comme victoires » reste négative), mais les taux élevés de troncature du graphe signifient que son score contre l’adversaire aléatoire est mesuré sur moins de parties décidées (43–80 par graine).

7.4 Validité statistique (graines, dépendances)

Cinq graines par bras bornent les statistiques — l’extension pré-engagée de trois à cinq graines (D-031) a resserré quatre des six intervalles et absorbé les graines les plus favorables au graphe observées sans changer le verdict ; le contraste B-MCTS reste indiscernable de zéro sous les deux lectures. Le rejet ne repose pas sur un seul intervalle : il repose sur la cohérence entre graines à travers deux adversaires et deux lectures simultanément, plus l’analyse des bornes. Les ouvertures sont partagées entre les bras (l’appariement est respecté dans le plan d’expérience) ; le contraste entre bras applique le bootstrap aux graines, non aux parties ; aucune pseudo-réplication au niveau des parties n’entre dans aucun intervalle.

7.5 Validité externe (variante, matériel, budget)

Une variante (Hive de base), une machine, un petit budget (10 générations $\times$ 500 parties ; $\sim$ 18–48 h/exécution), auto-jeu en régime précoce tout du long. L’asymétrie de fournisseur (CoreML favorise le réseau convolutif ; le GNN riche en opérations de collecte (gather) s’exécute le plus vite sur CPU) est une propriété authentique du matériel de déploiement, rapportée et imputée — sur d’autres accélérateurs, la lecture à temps mural pourrait se déplacer. Rien ici ne se généralise à d’autres jeux, de plus grands budgets ou des architectures de graphe plus riches ; l’étude répond à sa question pré-enregistrée à l’intérieur de son périmètre pré-enregistré, et la réponse négative est le produit : le choix d’encodage importe (comme AZ-Hive l’a constaté), et pour Hive à petit budget il favorise la grille.

Copie française (D-006) — version 1.0-draft 2026-10-09; les nombres et affirmations sont identiques au master anglais.

Chapitre 8 — Méthodologie de travail : une recherche humain-IA à portes (livret, 2026-10-09)

Condensé de docs/methodology.md (espace de travail, v1.0) et du journal méthodologique en ajout seul ; ce chapitre est aussi la déclaration d'usage de l'IA du rapport, sous forme développée.

Division du travail

Cette étude a été exécutée sous **délégation au niveau des objectifs avec autorité humaine mécanique**. L'auteur humain est le chercheur principal : il détient les questions de recherche, chaque engagement scientifique, chaque dépense, tout ce qui est public, et le dernier mot sur chaque affirmation — une responsabilité qui n'est pas délégable. L'assistant IA (Claude, Anthropic — opérant en sessions Claude Code) est l'exécutant (*runtime*) : à partir du plan de recherche, il se route lui-même, construit, mesure, journalise et rédige vers l'état final du plan, sans instruction tâche par tâche.

La frontière est imposée par six **portes** énumérant les décisions que seul l'humain prend : geler un protocole, une partition ou un ensemble de test (G-FREEZE) ; dépenser ou lancer un calcul long (G-SPEND) ; tout ce qui quitte la machine (G-PUBLIC) ; la réutilisation de matériel aux droits non établis (G-RIGHTS) ; le contact institutionnel (G-ADMIN) ; la destruction de données ou de résultats (G-DESTRUCTIVE). Chaque franchissement de porte dans cette étude est consigné dans le journal des décisions avec l'approbation de l'humain citée verbatim — le gel du protocole, le gel de la population d'adversaires avec son sous-choix budgétaire, le gel des ouvertures, quatre approbations de calcul avec dimensionnement explicite, et les pré-engagements d'extension de graines.

Pourquoi l'état vit dans des fichiers

Les sessions sont sans état par conception : une session lit le fichier de routage, exécute le document de pipeline de la phase active (tâches ordonnées, chacune avec une vérification d'acceptation, closes par des critères de sortie), journalise tout ce qui est mesuré, et met à jour les fichiers d'état en dernier. La qualité de la recherche est ainsi une propriété d'**artefacts de processus** — les journaux, le journal des décisions, les documents gelés avec leurs hachages, le registre des affirmations — et non de la mémoire ou de la compétence d'une session. Tout ce que contient ce livret remonte à ces artefacts.

Ce que la discipline a attrapé

Le journal méthodologique consigne chaque incident où la discipline a changé une issue. Pendant cette étude, elle a attrapé, entre autres : un interblocage de protocole dans les documents de pipeline avant toute exécution ; un désaccord moteur-contre-corpus résolu *contre* le corpus écrit à la main (le moteur avait raison, et la trace de l'erreur a été conservée) ; un défaut du harnais d'évaluation détecté parce que trois adversaires « différents » produisaient des résultats identiques ; un chemin de perte silencieuse d'enregistrements dans le lanceur de matchs, trouvé grâce à la règle selon laquelle chaque position d'échec sélectionnée doit être *reproduite et vérifiée* avant publication ; et une ablation qui avait silencieusement divergé vers NaN pendant dix générations, attrapée par la même alarme de résultats identiques et convertie en la découverte d'ablation la plus nette de l'étude. Le motif est l'affirmation centrale de la méthodologie : **à petite échelle, l'erreur de harnais est une menace plus grande que le bruit statistique, et seule la vérification mécanique l'attrape.**

Ce que l'IA n'a pas fait

L'IA n'a choisi aucune hypothèse, n'a rien gelé, n'a rien dépensé, n'a rien publié et n'a décidé d'aucune affirmation. Là où ses brouillons contenaient des erreurs, le processus — passes de traduction, vérifications mécaniques de cohérence, relectures adversariales — en a fait remonter plusieurs (comptes de graines périmés, limites d'affirmations périmées) avant cette version ; le dossier de contrôle final liste les vérifications. L'auteur humain a personnellement vérifié les conclusions qu'il signe.

Copie française (D-006) — version 1.0-draft 2026-10-09; les nombres et affirmations sont identiques au master anglais.

Chapitre 9 — Conclusion (brouillon du livret, 2026-10-09)

Aucun résultat nouveau ici, conformément au plan.

À l'intérieur du périmètre testé — Hive en jeu de base, un réseau relationnel simple à passage de messages à capacité appariée contre un CNN grille, dix générations d'auto-jeu à petit budget, cinq graines par bras, une population gelée de trois adversaires — **la réponse à RQ-H1 est non** : la représentation en graphe n'a pas appris une meilleure politique que la représentation en grille, ni sous la lecture à exemples égaux ni sous la lecture à temps mural égal, et la règle de rejet pré-enregistrée s'est déclenchée exactement telle que gelée. Le déficit est le plus grand là où Hive est le plus tactique, le bras graphe paie deux fois le temps mural, et son mode de défaillance — gagner du matériel sans convertir — est visible précisément parce que la troncature n'a jamais été repliée en nulles. Le résultat ne contredit pas les constats positifs pour les graphes sur Hex et aux échecs ; il les borne : là où la tactique à courte portée décide des parties et où les budgets sont petits, les artefacts de cadre coûtent moins cher que l'absence de cadre.

Deux suites sont motivées par les données plutôt que par l'espoir. Premièrement, **la pathologie de conversion est un défaut ciblable** : la tête de valeur du bras graphe apprend honorablement tandis que sa politique échoue sur les séquences forcées, ce qui suggère une expérience sur des remèdes au moment de la recherche (budgets d'évaluation plus profonds dans les positions gagnées, ou cibles auxiliaires pour les coups forçants) sous la même évaluation gelée — un changement à un seul composant du pipeline partagé, applicable aux deux bras. Deuxièmement, **le constat de stabilité mérite d'être isolé** : A1/A1' ont montré que les relations typées par direction importent surtout pour l'optimisation à cette échelle ; une étude contrôlée des choix de normalisation et d'échelle de gradient pour des couches de graphe à relations partagées pourrait découpler l'entraînabilité du contenu représentationnel — et dirait si l'adjacence naïve, correctement stabilisée, est véritablement suffisante pour Hive.

La méthode plus large tient indépendamment du signe du résultat : le pré-enregistrement avec artefacts gelés, les doubles lectures de budget, la troncature comme issue, l'inférence au niveau des graines et la rédaction au fil de l'eau ont transformé une réponse négative en un objet scientifique utilisable sur un seul ordinateur portable.

Bibliography and webographie (generated — plan ch. 27)

Generated by `scripts/make_bibliography.py` from the reading notes; every entry was actually consulted (its note is the evidence) and no entry appears in both sections.

Bibliography

- Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, Marc G. Bellemare. “Deep Reinforcement Learning at the Edge of the Statistical Precipice. NeurIPS 2021 (peer-reviewed; Outstanding Paper Award); arXiv:2108.13264. arXiv:2108.13264 (v1 Aug 30, 2021; v4 Jan 5, 2022), DOI 10.48550/arXiv.2108.13264; NeurIPS 2021 proceedings. Code: google-research/rliable (pip package rliable). (note: `docs/reading/agarwal-2021-precipice.md`, read 2026-09-09)
- Shai Ben-Assayag, Ran El-Yaniv. “Train on Small, Play the Large: Scaling Up Board Games with AlphaZero and GNN. arXiv preprint 2021 (arXiv:2107.08387 [cs.LG], v1 submitted 18 Jul 2021; no journal/conference version found as of 2026-09-26). arXiv:2107.08387v1 (18 Jul 2021), DOI 10.48550/arXiv.2107.08387; no code commit (code unreleased) (note: `docs/reading/benassayag-2021-scalable-alphazero.md`, read 2026-09-26)
- Tristan Cazenave, Yen-Chi Chen, Guan-Wei Chen, Shi-Yu Chen, Xian-Dong Chiu, Julien Dehos, Maria Elsa, Qucheng Gong, Hengyuan Hu, Vasil Khalidov, Cheng-Ling Li, Hsin-I Lin, Yu-Jin Lin, Xavier Martinet, Vegard Mella, Jeremy Rapin, Baptiste Roziere, Gabriel Synnaeve, Fabien Teytaud, Olivier Teytaud, Shi-Cheng Ye, Yi-Jun Ye, Shi-Jim Yen, Sergey Zagoruyko. “Polygames: Improved Zero Learning. ICGA Journal 42(4), pp. 244-256, 2020 (issue published Jan 2021); arXiv:2001.09832 (Jan 2020). DOI 10.3233/ICG-200157; arXiv:2001.09832; code repo archived 2022-03-02 (no pinned release cited) (note: `docs/reading/cazenave-2020-polygames.md`, read 2026-09-09)
- Danilo de Goede, Duncan Kampert, Ana Lucia Varbanescu. “The Cost of Reinforcement Learning for Game Engines: The AZ-Hive Case-study. ICPE 2022 (13th ACM/SPEC International Conference on Performance Engineering, Beijing), pp. 145-152. DOI 10.1145/3489525.3511685; open PDF at https://research.spec.org/icpe_proceedings/2022/proceedings/p145.pdf; BeeKeeper engine repo commit not pinned in paper (note: `docs/reading/degoede-2022-azhive.md`, read 2026-09-09)
- William L. Hamilton, Rex Ying, Jure Leskovec. Inductive Representation Learning on Large Graphs. NeurIPS 2017 (peer-reviewed; NIPS at the time). arXiv:1706.02216.. arXiv:1706.02216 (v4, 2018-09-10), DOI 10.48550/arXiv.1706.02216; NeurIPS 2017 proceedings. Code: github.com/williamleif/GraphSAGE (no specific release pinned). (note: `docs/reading/hamilton-2017-graphsage.md`, read 2026-09-09)
- Andy L. Jones. “Scaling Scaling Laws with Board Games. arXiv preprint 2021 (arXiv:2104.03113; v1 Apr 7, 2021, v2 Apr 15, 2021; not peer-reviewed). arXiv:2104.03113v2, DOI 10.48550/arXiv.2104.03113. Code/data: boardlaw project (andyljones.com/boardlaw, github.com/andyljones/boardlaw). (note: `docs/reading/jones-2021-scaling.md`, read 2026-09-09)
- Duncan Kampert, Ana-Lucia Varbanescu, Matthias Müller-Brockhausen, Aske Plaat. “Mimicking the Human Approach in the Game of Hive.” (preprint circulated as “Better AI for Hive: Mimicking human game-play strategies”) IEEE SSCI 2021 (IEEE Symposium Series on Computational Intelligence, Orlando, Dec 2021). IEEE Xplore document 9659999

(<https://ieeexplore.ieee.org/document/9659999/>); preprint PDF:
https://liacs.leidenuniv.nl/~plaata1/papers/IEEE_Conference_Hive_D_Kampert.pdf; dblp:
conf/ssci/KampertVMP21 (note: *docs/reading/kampert-2021-mimicking-hive.md*, read 2026-09-09)

- Yannik Keller, Jannis Blüml, Gopika Sudhakaran, Kristian Kersting. From Images to Connections: Can DQN with GNNs learn the Strategic Game of Hex? arXiv preprint 2023 (arXiv:2311.13414; submitted to ICLR via OpenReview id dYaeDrazj5, not listed as accepted — treat as non-peer-reviewed preprint).. arXiv:2311.13414 (2023-11-22), <https://arxiv.org/abs/2311.13414>; OpenReview forum dYaeDrazj5. Code: github.com/yannikkellerde/GNN_Hex (no release pinned). (note: *docs/reading/keller-2023-graphdqn-hex.md*, read 2026-09-09)
- Tomas Rigaux, Hisashi Kashima. “Enhancing Chess Reinforcement Learning with Graph Representation. NeurIPS 2024 (Advances in Neural Information Processing Systems 37, main conference track); arXiv:2410.23753. arXiv:2410.23753v1 (31 Oct 2024), DOI 10.48550/arXiv.2410.23753; NeurIPS proceedings DOI 10.52202/079017-0006; code repo not pinned to a commit in the paper (note: *docs/reading/rigaux-2024-chess-graph-rl.md*, read 2026-09-26)
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, Demis Hassabis. “Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm.” Also published as: “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play,” Science 362(6419):1140-1144, 2018. arXiv preprint 2017 (arXiv:1712.01815, v1 Dec 5, 2017); peer-reviewed version in Science 2018 (vol. 362, issue 6419, pp. 1140-1144). arXiv:1712.01815v1, DOI 10.48550/arXiv.1712.01815; Science version DOI 10.1126/science.aar6404. No official code release. (note: *docs/reading/silver-2017-alphazero.md*, read 2026-09-09)
- Elise van der Pol, Daniel E. Worrall, Herke van Hoof, Frans A. Oliehoek, Max Welling. MDP Homomorphic Networks: Group Symmetries in Reinforcement Learning. NeurIPS 2020 (peer-reviewed). arXiv:2006.16908.. arXiv:2006.16908 (v2, 2021-01-20), <https://arxiv.org/abs/2006.16908>; NeurIPS 2020 proceedings <https://papers.nips.cc/paper/2020/hash/2be5f9c2e3620eb73c2972d7552b6cb5-Abstract.html>. Code: github.com/ElisevanderPol/mdp-homomorphic-networks (no release pinned). (note: *docs/reading/vanderpol-2020-mdp-homomorphic.md*, read 2026-09-09)
- Oriol Vinyals, Meire Fortunato, Navdeep Jaitly. Pointer Networks. NeurIPS (NIPS) 2015, Advances in Neural Information Processing Systems 28 (peer-reviewed). arXiv:1506.03134.. arXiv:1506.03134 (v2, 2017-01-02), <https://arxiv.org/abs/1506.03134>; NIPS 2015 proceedings https://proceedings.neurips.cc/paper_files/paper/2015/hash/29921001f2fo4bd3baee84a12e98098-Abstract.html. (note: *docs/reading/vinyals-2015-pointer-networks.md*, read 2026-09-09)
- David J. Wu. “Accelerating Self-Play Learning in Go. arXiv preprint (arXiv:1902.10565; v1 Feb 27, 2019, v5 Nov 9, 2020); presented at the AAAI-20 Workshop on Reinforcement Learning in Games (not a full peer-reviewed proceedings paper). arXiv:1902.10565v5, DOI 10.48550/arXiv.1902.10565. Code: github.com/lightvector/KataGo (our reference reading: v5 paper; cite a specific release tag if we import any technique implementation). (note: *docs/reading/wu-2020-katago.md*, read 2026-09-09)

Webographie

- Jan Pfeifer (GitHub user janpfeifer). “hiveGo — Go Implementation of Hive Game.” GitHub repository. GitHub repository, 2018–2026 (created 2018-08-24, originally TensorFlow; refreshed 2025 onto GoMLX; last push 2026-08-20). main branch, commit d6ff95418d2b (2026-08-20,

“Updated main.wasm version.”); <https://github.com/janpfeifer/hiveGo>; consulted 2026-09-26
(note: *docs/reading/pfeifer-2025-hivego.md*, read 2026-09-26)

*Copie française (D-006) — version 1.0-draft 2026-10-09; les nombres et affirmations sont identiques
au master anglais.*

Annexe A — Corpus de positions annotées à la main

Chaque attendu ci-dessous a été écrit à la main à partir des règles de l'éditeur AVANT toute exécution du moteur (oracle avant toute sortie de modèle) ; l'exécution du moteur qui a suivi est journalisée, et l'unique désaccord constaté a été résolu contre le corpus (une erreur de transit One-Hive dans une séquence de mise en place), le moteur ayant eu raison. Ces rendus sont générés à partir des fichiers de cas exécutables par `scripts/make_annex_corpus.py` — les tests et l'annexe ne peuvent pas diverger.

A.1 Corpus critique de règles (30 cas, H2)

Cas de correction des règles : placement, glissement/liberté de mouvement, portes, empilement, One-Hive, états terminaux, passe forcée, et cas d'étourdissement (stun) du Pillbug comme garde du noyau.

Coo1 — La Reine ne peut pas être placée au premier tour (règle de tournoi)

- **Règle** : Règle d'ouverture de tournoi (*Tournament variant of the official rules (README source 4); Gen42 2010 rulesheet p. 3 alone would allow it*)
- **Mise en place** : ``
- **Attente** : `move_illegal {'move': 'wQ'}`
- **Justification manuscrite** : La feuille de règles de base 2010 dit que la Reine « peut être placée à tout moment de votre premier à votre quatrième tour » (p. 3), mais la variante de tournoi — adoptée par l'UHP et par les deux moteurs de référence, et la convention que cette étude fixe — interdit de placer la Reine au premier tour de l'un ou l'autre joueur. Le premier coup des Blancs « wQ » doit donc être rejeté.

Coo2 — La Reine doit être placée au quatrième tour si elle ne l'a pas été avant

- **Règle** : Placement de votre Reine (*Gen42 Hive rulesheet p. 3*)
- **Mise en place** : `wS1;bS1 wS1-;wG1 -wS1;bG1 bS1-;wA1 -wG1;bA1 bG1-`
- **Attente** : `all_moves_place {'piece': 'wQ'}`
- **Justification manuscrite** : « Vous devez placer votre Reine à votre quatrième tour si vous ne l'avez pas placée avant. » (p. 3). C'est le quatrième tour des Blancs et wQ est encore en réserve, donc chaque coup légal doit être un placement de wQ. (Les coups de déplacement sont de plus exclus par la règle de Déplacement, p. 3 : aucun déplacement avant que la reine soit placée.)
Légalité de la mise en place : chaque placement blanc ne touche que des pièces blanches, chaque placement noir que des pièces noires (Placement, p. 2).

Coo3 — Aucune pièce ne peut se déplacer avant que la reine de ce joueur soit placée

- **Règle** : Déplacement (*Gen42 Hive rulesheet p. 3*)
- **Mise en place** : `wS1;bS1 wS1-`
- **Attente** : `all_moves_are_placements`
- **Justification manuscrite** : « Une fois votre Reine placée (mais pas avant), vous pouvez décider d'utiliser chaque tour suivant pour placer une autre tuile ou pour déplacer l'une des pièces déjà placées. » (p. 3). La reine des Blancs n'est pas placée au tour 2, donc wS1 ne doit avoir aucun coup de déplacement — seuls des placements sont proposés.

Coo4 — Après les premières pièces, les placements ne peuvent pas toucher la couleur adverse

- **Règle :** Placement (*Gen42 Hive rulesheet p. 2*)
- **Mise en place :** wG1;bS1 wG1/
- **Attente :** moves_for_piece {'piece': 'wB1', 'moves': ['wB1 wG1\\', 'wB1 /wG1', 'wB1 -wG1']}
- **Justification manuscrite :** « ...à l'exception de la première pièce placée par chaque joueur, les pièces ne peuvent pas être placées à côté d'une pièce de la couleur de l'adversaire. » (p. 2). wG1 est à l'origine avec bS1 à son nord-est. Des cinq voisines vides de wG1 (E, SE, SO, O, NO), les cellules E et NO touchent chacune aussi bS1 (ce sont les deux cellules adjacentes à la fois à wG1 et à sa voisine NE), donc une nouvelle pièce blanche ne peut aller qu'au SE, au SO ou à l'O de wG1. Placements attendus pour wB1 : exactement ces trois cellules. Géométrie à la main : axial E=(1,0), NE=(1,-1) ; les voisines de la cellule NE (1,-1) comprennent (1,0)=E-de-l'origine et (0,-1)=NO-de-l'origine.

Coo5 — La première pièce du second joueur rejoint la première pièce (contact ennemi permis)

- **Règle :** Déroulement de la partie / Placement (*Gen42 Hive rulesheet pp. 2-3*)
- **Mise en place :** wS1
- **Attente :** moves_for_piece {'piece': 'bG1', 'moves': ['bG1 wS1-', 'bG1 wS1/', 'bG1 wS1\\', 'bG1 -wS1', 'bG1 /wS1', 'bG1 \\wS1']}
- **Justification manuscrite :** « La partie commence par un joueur qui place une pièce de sa main au centre de la table, puis le joueur suivant joint l'une de ses propres pièces à celle-ci, bord à bord. » (p. 2) — l'exception de la première pièce à la règle de placement sur sa propre couleur. La première pièce des Noirs doit rejoindre wS1 bord à bord, donc bG1 peut être placée sur n'importe laquelle des six cellules adjacentes à wS1, et nulle part ailleurs.

Coo6 — Reine à la pointe de la ruche : exactement les deux glissements qui gardent le contact

- **Règle :** Reine / Liberté de mouvement / One-Hive (contact) (*Gen42 Hive rulesheet pp. 4, 9, 10*)
- **Mise en place :** wS1;bS1 wS1-;wQ -wS1;bQ bS1-
- **Attente :** moves_for_piece {'piece': 'wQ', 'moves': ['wQ \\wS1', 'wQ /wS1']}
- **Justification manuscrite :** La Reine « ne peut se déplacer que d'un espace par tour » (p. 4) en un mouvement de glissement (p. 10), et « toutes les pièces doivent toujours toucher au moins une autre pièce » (p. 3 NB). wQ est à la pointe ouest d'une ligne droite de quatre. De ses cinq voisines vides, seules les deux cellules qui sont aussi adjacentes à sa voisine wS1 (les cellules au NO et au SO de wS1) gardent le contact avec la ruche après le glissement ; les trois cellules plus à l'ouest ne touchent plus rien une fois la reine partie. Aucune des deux destinations n'est derrière une porte (pour chaque glissement, des deux cellules flanquantes, l'une est occupée, l'autre vide). Attendu : exactement ces deux coups.

Coo7 — Fourmi enfermée dans une poche : la seule sortie est une porte, donc elle ne peut pas bouger

- **Règle :** Liberté de mouvement (*Gen42 Hive rulesheet p. 10*)
- **Mise en place :** wA1;bS1 wA1-;wQ \\wA1;bQ bS1-;wG1 -wA1;bB1 bQ/;wS1 /wA1;bB1 bS1/;wB1 \\wQ;bB1 wA1/

- **Attente :** `moves_for_piece {'piece': 'wA1', 'moves': []}`
- **Justification manuscrite :** « Si une pièce est entourée au point de ne plus pouvoir physiquement glisser hors de sa position, elle ne peut pas être déplacée. » (p. 10). Cinq des six voisines de la fourmi sont occupées. La seule voisine vide (au SE de la fourmi) est flanquée par bS1 (à l'E de la fourmi) et wS1 (au SO de la fourmi) — les deux cellules adjacentes à la fois à la fourmi et à cet espace — donc la fourmi ne peut pas physiquement y glisser. Retirer la fourmi ne scinderait PAS la ruche (l'anneau bB1-wQ-wG1-wS1 plus bS1 reste connexe), donc le blocage relève purement de la liberté de mouvement, pas de One-Hive. La fourmi, normalement la pièce la plus mobile, a zéro coup légal.

Coo8 — L'araignée se déplace d'exactly trois espaces le long du bord de la ruche — deux destinations

- **Règle :** Araignée (*Gen42 Hive rulesheet* p. 7)
- **Mise en place :** `wG1;bS1 wG1-;wQ -wG1;bQ bS1-;wS1 \wQ;bG1 bQ-`
- **Attente :** `moves_for_piece {'piece': 'wS1', 'moves': ['wS1 bS1/', 'wS1 wQ\\']}`
- **Justification manuscrite :** « L'Araignée se déplace de trois espaces par tour — ni plus, ni moins. Elle doit suivre un chemin direct et ne peut pas revenir sur ses pas. Elle ne peut se déplacer qu'autour des pièces avec lesquelles elle est en contact direct à chaque pas. » (p. 7). La ruche moins l'araignée est une ligne droite de cinq pièces dont la frontière est un unique anneau de 14 cellules sans portes ; chaque cellule de l'anneau touche la ligne, et les cellules hors de l'anneau ne touchent rien (exclues par l'exigence de contact). Depuis sa position sur l'anneau, l'araignée a donc exactement deux marches de trois pas — trois cellules dans le sens horaire et trois cellules dans le sens antihoraire : la cellule au NE de bS1, et la cellule au SE de wQ. Les arrêts après un ou deux pas sont exclus (« ni moins »), le retour en arrière est exclu.

Coo9 — Sauterelle : ne saute que le long de rangées occupées, pas de glissements d'un espace

- **Règle :** Sauterelle (*Gen42 Hive rulesheet* p. 6)
- **Mise en place :** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 \wS1;bG1 bQ-`
- **Attente :** `moves_for_piece {'piece': 'wG1', 'moves': ['wG1 wS1\\', 'wG1 /wQ']}`
- **Justification manuscrite :** « Elle saute depuis son espace par-dessus un nombre quelconque de pièces (mais au moins une) jusqu'au premier espace inoccupé le long d'une rangée droite de pièces jointes. » (p. 6). La sauterelle touche des cellules occupées dans exactement deux de ses six directions : SE (par-dessus wS1, atterrissant dans l'espace suivant, au SE de wS1) et SO (par-dessus wQ, atterrissant au SO de wQ). Dans les quatre autres directions, la cellule adjacente est vide, et un saut « par-dessus au moins une » pièce est impossible — en particulier, les quatre cellules vides adjacentes ne sont PAS des destinations : la sauterelle « ne se déplace pas autour de l'extérieur de la Ruche comme les autres créatures ». Attendu : exactement les deux cellules d'atterrissage.

Co10 — La sauterelle saute une rangée complète de cinq pièces jusqu'au premier espace vide

- **Règle :** Sauterelle (*Gen42 Hive rulesheet* p. 6)
- **Mise en place :** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 -wQ;bG1 bQ-`
- **Attente :** `moves_for_piece {'piece': 'wG1', 'moves': ['wG1 bG1-']}`
- **Justification manuscrite :** « ...par-dessus un nombre quelconque de pièces (mais au moins une) jusqu'au premier espace inoccupé le long d'une rangée droite de pièces jointes. » (p. 6). Plein

est, la sauterelle fait face à la rangée ininterrompue wQ, wS1, bS1, bQ, bG1 ; le premier espace inoccupé au-delà est la cellule à l'E de bG1 — l'unique destination. Elle doit atterrir là, pas avant (chaque cellule plus proche dans la rangée est occupée). Dans les cinq autres directions, la cellule adjacente est vide, donc aucun saut n'existe. La sauterelle est une feuille de la ruche, donc One-Hive ne la restreint pas.

Co11 — La seule voisine ouverte de la reine est derrière une porte : zéro coup

- **Règle :** Liberté de mouvement (*Gen42 Hive rulesheet p. 10*)
- **Mise en place :** wS1;bS1 -wS1;wB1 wS1/;bQ -bS1;wQ wB1-;bG1 \bQ;wS2 wQ/;bA1 /bQ;wG1 -wS2;bS2 /bS1;wA1 wS2\;bB1 \bG1;wG2 wQ\;bG2 \bB1
- **Attente :** moves_for_piece {'piece': 'wQ', 'moves': []}
- **Justification manuscrite :** « De même, aucune pièce ne peut se déplacer dans un espace où elle ne peut pas physiquement glisser. » (p. 10). Cinq des six voisines de la reine sont des pièces blanches ; la sixième (la cellule à l'O de wG2, également au SE de wB1) est vide, mais les deux cellules adjacentes à la fois à la reine et à cet espace sont wG2 et wB1 — toutes deux occupées — donc la reine ne peut pas physiquement y glisser. Retirer la reine laisse le fer à cheval blanc wS1-wB1-wG1-wS2-wA1-wG2 connexe (et la chaîne noire pend de wS1 via bS1), donc One-Hive autoriserait le coup ; le blocage relève purement de la liberté de mouvement. Attendu : la reine n'a aucun coup légal.

Co12 — La fourmi atteint chaque cellule du périmètre de la ruche (13 destinations)

- **Règle :** Fourmi soldat / Liberté de mouvement (*Gen42 Hive rulesheet pp. 8, 10*)
- **Mise en place :** wG1;bS1 wG1-;wQ -wG1;bQ bS1-;wA1 \wQ;bG1 bQ-
- **Attente :** moves_for_piece {'piece': 'wA1', 'moves': ['wA1 -wQ', 'wA1 \wG1', 'wA1 \bS1', 'wA1 \bQ', 'wA1 \bG1', 'wA1 bG1/', 'wA1 bG1-', 'wA1 bG1\\', 'wA1 bQ\\', 'wA1 bS1\\', 'wA1 wG1\\', 'wA1 wQ\\', 'wA1 /wQ']}
- **Justification manuscrite :** « La Fourmi soldat peut se déplacer de sa position vers n'importe quelle autre position autour de la Ruche, pourvu que les restrictions soient respectées. » (p. 8). La ruche moins la fourmi est une ligne droite de cinq pièces ; sa frontière est un unique anneau de 14 cellules sans portes (chaque pas de glissement est flanqué d'une cellule de la ligne et d'une cellule vide), et chaque cellule de l'anneau touche la ligne. La fourmi part de l'anneau, sur la cellule au NO de wQ, donc elle peut s'arrêter sur n'importe laquelle des 13 autres cellules de l'anneau : le bout ouest (à l'O de wQ), les cinq cellules de l'épaule nord (au NO de chaque pièce de la ligne plus au NE de bG1), le bout est (à l'E de bG1), et les six cellules de l'épaule sud (au SE de chaque pièce de la ligne plus au SO de wQ). Les cellules hors de l'anneau ne touchent aucune pièce et sont exclues (p. 3 NB : les pièces doivent toujours toucher au moins une autre pièce).

Co13 — Scarabée au sol : deux glissements et deux escalades

- **Règle :** Scarabée (*Gen42 Hive rulesheet pp. 4-5, 10*)
- **Mise en place :** wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bG1 bQ-
- **Attente :** moves_for_piece {'piece': 'wB1', 'moves': ['wB1 wS1', 'wB1 wQ', 'wB1 \bS1', 'wB1 \wQ']}
- **Justification manuscrite :** « Le Scarabée, comme la Reine, ne se déplace que d'un espace par tour. Contrairement à toute autre créature cependant, il peut aussi se déplacer sur le dessus de la Ruche. » (p. 4). Depuis (au NO de wS1), le scarabée peut grimper sur l'une ou l'autre des pièces adjacentes — wS1 ou wQ — ou glisser au sol vers les deux cellules vides qui gardent le contact avec

la ruche : au NO de bS1 (touchant wS1 et bS1) et au NO de wQ (touchant wQ). Les deux voisines vides restantes ne touchent aucune pièce après que le scarabée se soulève, donc elles sont exclues (p. 3 NB). Aucune porte ne bloque aucun des quatre coups (chacun est flanqué d'au plus une cellule occupée, et pour les escalades les piles flanquantes ne sont pas plus hautes que la destination). Exactement quatre coups — ce qui correspond au compte de l'exemple de scarabée de la feuille de règles elle-même.

Co14 — Scarabée au sommet de la ruche : les six cellules voisines

- **Règle :** Scarabée (*Gen42 Hive rulesheet p. 5; beetle-gate ruling, World Hive Tournaments Rules FAQ*)
- **Mise en place :** wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bG1 bQ-;wB1 wS1;bA1 bG1-
- **Attente :** moves_for_piece {'piece': 'wB1', 'moves': ['wB1 bS1', 'wB1 wQ', 'wB1 \\wS1', 'wB1 \\bS1', 'wB1 wS1\\', 'wB1 wQ\\']}
- **Justification manuscrite :** « Depuis sa position au sommet de la Ruche, le Scarabée peut se déplacer de tuile en tuile sur le dessus de la Ruche. Il peut aussi descendre dans des espaces entourés et donc inaccessibles à la plupart des autres créatures. » (p. 5). Posé sur wS1, le scarabée peut aller sur chacune des six cellules voisines : passer sur bS1 ou wQ (deux piles de hauteur 1), ou descendre sur n'importe laquelle des quatre cellules vides autour de wS1 — chacune touchant encore wS1 lui-même, donc le contact tient. Aucune paire de piles flanquantes n'est plus haute à la fois que l'origine (hauteur 1 sous le scarabée) et la destination, donc aucune porte du scarabée ne s'applique (FAQ). One-Hive ne peut pas être violée : wS1 reste où il est. Exactement six destinations.

Co15 — Porte du scarabée : la descente entre deux piles de hauteur 2 est bloquée

- **Règle :** Liberté de mouvement au-dessus du sol (porte du scarabée) (*World Hive Tournaments Rules FAQ; Gen42 Hive rulesheet p. 10*)
- **Mise en place :** wS1;bG1 wS1-/wQ /wS1;bQ bG1-/wG1 wS1\;bB1 bQ-/wB1 -wS1;bB1 bQ/wB2 /wQ;bB1 bG1/wB1 wS1;bQ bB1-/wB2 wQ;bQ bB1-/wB2 wG1;bA1 bQ/
- **Attente :** move_illegal {'move': 'wB1 bB1\\'}
- **Justification manuscrite :** « Quand une pièce monte ou descend la ruche, ou se déplace en restant au sommet de la ruche, elle doit pouvoir glisser selon la règle de liberté de mouvement, qui s'applique aux niveaux supérieurs au sol. Si deux piles forment une porte au-dessus du niveau du sol (nous l'appelons porte du scarabée), les pièces ne peuvent pas s'y glisser. » (WHT Rules FAQ). wB1 est posé sur wS1 (son propre niveau : au sommet d'une pièce de hauteur 1) ; la cellule cible au SE de la pile bB1 est vide (hauteur 0). Les deux cellules adjacentes à la fois à l'origine et à la cible portent les piles bG1+bB1 et wG1+wB2, toutes deux de hauteur 2 — strictement plus hautes à la fois que l'origine sans le scarabée (1) et que la destination (0) — donc le scarabée ne peut pas glisser vers le bas entre elles. La descente doit être rejetée. (One-Hive l'autoriserait : wS1 reste en place ; le contact tient via les piles flanquantes.)

Co16 — Une pièce surmontée d'un scarabée ne peut pas bouger

- **Règle :** Scarabée (immobilité sous pile) (*Gen42 Hive rulesheet p. 5*)
- **Mise en place :** wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bG1 bQ-;wB1 wS1;bA1 bG1-
- **Attente :** move_illegal {'move': 'wS1 bS1\\'}
- **Justification manuscrite :** « Une pièce avec un scarabée sur elle est incapable de bouger » (p. 5). wS1 est sous wB1, donc toute tentative de déplacer wS1 — ici un coup d'araignée vers la

cellule au SE de bS1 — doit être rejetée, que le chemin soit par ailleurs légal ou non pour une araignée.

Co17 — La pile prend la couleur du scarabée : les Blancs peuvent placer à côté d'une reine noire recouverte

- **Règle :** Scarabée (couleur de la pile) / Placement (*Gen42 Hive rulesheet pp. 2, 5*)
- **Mise en place :** wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bA1 bS1\;wB1 \bS1;bG1 bA1\;wB1 \bQ;bG2 bG1\;wB1 bQ;bB1 bG2\
- **Attente :** move_legal {'move': 'wG1 wB1-'}
- **Justification manuscrite :** « ...aux fins des règles de placement de la p. 2, la pile prend la couleur du Scarabée. » (p. 5). Le scarabée blanc est posé sur la reine noire à l'extrémité est de la ruche. La cellule à l'E de cette pile ne touche aucune autre pièce, donc un placement blanc à cet endroit n'est adjacent qu'à une pile dont la couleur est — par la règle — blanche. Le placement de wG1 à cet endroit doit être accepté. (Sans la règle de couleur de pile, la cellule serait adjacente à une pièce noire et le placement serait illégal, p. 2.)

Co18 — La pile prend la couleur du scarabée : les Noirs ne peuvent PAS placer à côté de leur propre reine recouverte

- **Règle :** Scarabée (couleur de la pile) / Placement (*Gen42 Hive rulesheet pp. 2, 5*)
- **Mise en place :** wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bA1 bS1\;wB1 \bS1;bG1 bA1\;wB1 \bQ;bG2 bG1\;wB1 bQ;bB1 bG2\;wG1 -wQ
- **Attente :** move_illegal {'move': 'bB2 wB1-'}
- **Justification manuscrite :** Miroir de Co17 : la pile bQ+wB1 compte comme BLANCHE (« la pile prend la couleur du Scarabée », p. 5). La cellule à l'E de la pile ne touche que cette pile, donc pour les Noirs elle est adjacente à une pièce blanche et « les pièces ne peuvent pas être placées à côté d'une pièce de la couleur de l'adversaire » (p. 2). La tentative des Noirs de placer bB2 à cet endroit doit être rejetée — même si la pièce enfouie est la propre reine des Noirs.

Co19 — One-Hive : la seule connexion entre deux parties ne peut pas bouger

- **Règle :** Règle One-Hive (*Gen42 Hive rulesheet pp. 3, 9*)
- **Mise en place :** wS1;bS1 wS1-;wQ -wS1;bQ bS1-
- **Attente :** moves_for_piece {'piece': 'wS1', 'moves': []}
- **Justification manuscrite :** « Toutes les pièces doivent toujours toucher au moins une autre pièce. Si une pièce est la seule connexion entre deux parties de la Ruche, elle ne peut pas être déplacée. » (p. 3 NB) ; « Les pièces en jeu doivent être reliées à tout moment. À aucun moment vous ne pouvez laisser une pièce isolée (non reliée à la Ruche) ni séparer la Ruche en deux. » (p. 9). wS1 est le lien intérieur entre wQ d'un côté et bS1-bQ de l'autre : la soulever scinde la ruche, donc l'araignée n'a aucun coup légal — chaque destination, si valide soit-elle comme déplacement d'araignée, est exclue par One-Hive.

Co20 — Anneau : une pièce sur une boucle fermée peut bouger (pas un point d'articulation) ; l'œil de l'anneau est derrière une porte

- **Règle :** Règle One-Hive / Liberté de mouvement (*Gen42 Hive rulesheet pp. 9, 10*)
- **Mise en place :** wS1;bS1 -wS1;wG1 wS1/bQ -bS1;wQ wS1\;bG1 -bQ;wG2 wG1-;bG2 -bG1;wA1 wQ-;bA1 -bG2;wS2 wG2\;bB1 -bA1
- **Attente :** moves_for_piece {'piece': 'wQ', 'moves': ['wQ /wS1', 'wQ /wA1']}

- **Justification manuscrite** : Les six pièces blanches forment un anneau fermé, donc retirer wQ laisse les cinq autres connexes le long de la boucle (et la queue noire pend de wS1) : One-Hive permet à la reine de bouger. En glissant d'un espace (p. 4), la reine a trois voisines vides : l'œil de l'anneau et deux cellules extérieures. L'œil est flanqué par wS1 et wA1 — toutes deux occupées — donc la reine « ne peut pas se déplacer dans un espace où elle ne peut pas physiquement glisser » (p. 10). Les deux cellules extérieures (au SO de wS1, qui touche wS1 et bS1 ; et au SO de wA1, qui touche wA1) sont des glissements sans obstruction qui gardent le contact. Attendu : exactement ces deux destinations.

Co21 — La partie se termine quand une reine est complètement encerclée — même par sa propre couleur

- **Règle** : Le but de Hive / La fin de la partie (*Gen42 Hive rulesheet pp. 1, 11*)
- **Mise en place** : wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 -wQ;bG1 bQ-;wA1 -wG1;bG2 bQ-/;wA2 -wA1;bB1 \bQ;wA3 -wA2;bA1 bS1\;wS2 -wA3;bA2 bQ\
- **Attente** : game_over {'state': 'WhiteWins'}
- **Justification manuscrite** : « Les pièces entourant la Reine peuvent être un mélange de vos pièces et de celles de votre adversaire. » (p. 1). « La partie se termine dès qu'une Reine est complètement encerclée par des pièces de n'importe quelle couleur. La personne dont la Reine est encerclée perd la partie. » (p. 11). Le dernier placement des Noirs (bA2, au SE de leur propre reine) remplit la sixième et dernière cellule autour de bQ. Les pièces qui l'entourent sont toutes noires — sans importance selon la p. 1 — et c'est le propre coup des Noirs qui achève l'encercllement : les Noirs perdent, l'état de la GameString doit indiquer WhiteWins immédiatement après ce coup.

Co22 — Un seul coup encercle les deux reines simultanément : nulle

- **Règle** : La fin de la partie (*Gen42 Hive rulesheet p. 11*)
- **Mise en place** : wS1;bS1 wS1-/;wQ wS1\;bB1 bS1-;wA1 -wQ;bQ bB1\;wS2 /wQ;bQ /bB1;wG1 -wS2;bQ wQ-;wB1 -wA1;bA1 bQ\;wG1 wS2-;bG1 \bS1;wB1 \wA1;bA2 bQ-;wB1 \wS1;bA3 bB1\;wG2 -wB1;bG1 bS1\
- **Attente** : game_over {'state': 'Draw'}
- **Justification manuscrite** : « La personne dont la Reine est encerclée perd la partie, sauf si la dernière pièce à encercler sa Reine achève aussi l'encercllement de l'autre Reine. Dans ce cas, la partie est nulle. » (p. 11). Avant le dernier coup des Noirs, chaque reine a exactement une voisine vide — la même cellule (1,0), adjacente aux deux reines (les reines sont côte à côte, wQ à l'épaule NE de wS1, bQ à côté d'elle). La sauterelle noire en (1,-2) saute au SE par-dessus bS1 dans cette cellule, remplissant d'une seule pièce la sixième voisine des deux reines : la partie doit se terminer par une nulle (Draw), pas par une victoire de l'un ou l'autre camp.

Co23 — Un joueur qui ne peut ni placer ni bouger doit passer

- **Règle** : Impossibilité de déplacer ou de placer (*Gen42 Hive rulesheet pp. 2, 5, 10, 11*)
- **Mise en place** : wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bQ bS1-/;wA1 /wQ;bQ bS1-;wB1 \bS1;bQ bS1-/;wB1 bS1;bQ wB1-;wA1 /bQ;bQ wB1-/;wA2 /wQ;bQ wB1-;wA2 bQ\;bQ wB1-/;wA3 /wQ;bQ wB1-;wA3 bQ-;bQ wB1-/;wG1 /wB1;bQ wB1-;wG1 wB1/
- **Attente** : must_pass
- **Justification manuscrite** : « Si un joueur ne peut ni placer une nouvelle pièce ni déplacer une pièce existante, le tour passe à son adversaire, qui joue alors de nouveau. » (p. 11). Après le dernier coup des Blancs, les Noirs ont : bS1 sous wB1 — « une pièce avec un scarabée sur elle est

incapable de bouger » (p. 5), et la pile compte comme blanche pour le placement (p. 5) ; et bQ, dont les cinq voisines sont la pile, wG1, wA1, wA2 et wA3, sa seule voisine vide n'étant atteignable qu'entre wG1 et wA3 — une porte où la reine « ne peut pas physiquement glisser » (p. 10). Aucun placement n'est possible non plus : la seule cellule adjacente à une pièce à sommet noir est cette même cellule derrière la porte, qui touche aussi des pièces blanches (p. 2). Les Noirs ne sont pas perdus — bQ a une voisine vide, donc elle n'est pas encerclée — mais doivent passer.

Co24 — Capacité spéciale du Pillbug : déplacer une pièce amie adjacente (garde du noyau)

- **Règle** : Capacité spéciale du Pillbug (*Gen42 Pillbug rulesheet (English section)*)
- **Mise en place** : wP; bS1 wP-; wQ -wP; bQ bS1-
- **Attente** : move_legal {'move': 'wQ wP\\'}
- **Justification manuscrite** : « La capacité spéciale permet au Pillbug de déplacer une pièce adjacente (amie ou ennemie) de deux espaces ; vers le haut sur lui-même, puis vers le bas dans un autre espace vide adjacent à lui-même. » (feuille Pillbug). wQ est adjacente à wP ; la cellule cible au SE de wP est vide et adjacente à wP. Aucune des quatre exceptions ne s'applique : wQ ne vient pas d'être déplacée par l'autre joueur (le dernier coup des Noirs était le placement de bQ), wQ n'est pas dans une pile, retirer wQ ne scinde pas la ruche (c'est une feuille), et aucune pièce empilée ne forme de porte sur le chemin de montée et de descente. NOTE : ceci est un cas de garde du noyau étiqueté variante (protocole §2) — la variante de l'étude est le jeu de base ; les cas Pillbug ne protègent que le noyau de règles partagé.

Co25 — Une pièce que le Pillbug ennemi vient de déplacer est étourdie pour un tour (garde du noyau)

- **Règle** : Capacité spéciale du Pillbug (immobilité de la pièce déplacée) (*Gen42 Pillbug rulesheet (English section)*; *World Hive Tournaments Rules FAQ*)
- **Mise en place** : wS1; bP wS1-; wQ -wS1; bQ bP-; wA1 \\wQ; bG1 bQ-; wA1 \\bP; bG1 -wQ; wG1 -wA1; wA1 bP\\
- **Attente** : move_illegal {'move': 'wA1 bP/'}
- **Justification manuscrite** : « De plus, toute pièce déplacée par le Pillbug ne peut pas du tout être déplacée (directement ou via une action du Pillbug) au tour du joueur suivant. » (feuille Pillbug) ; FAQ : « toute pièce qui vient de bouger, au tour de l'autre joueur immédiatement après, est incapable de : bouger, être déplacée ou utiliser la capacité du pillbug. » Le Pillbug noir vient de lancer wA1 par-dessus lui-même jusqu'à la cellule au SE de bP (un usage légal : wA1 avait bougé pour la dernière fois deux plis plus tôt, donc l'exception du dernier-déplacé ne bloquait pas le lancer ; son retrait gardait la ruche entière puisque wG1 touche aussi wS1 et wQ). Au tour immédiatement suivant des Blancs, la fourmi lancée est étourdie : la tentative de coup de fourmi vers le NE de bP doit être rejetée. Cas de garde du noyau étiqueté variante (protocole §2).

Co26 — Le Pillbug ne peut pas déplacer la pièce que l'adversaire vient de déplacer (garde du noyau)

- **Règle** : Capacité spéciale du Pillbug (exceptions) (*Gen42 Pillbug rulesheet (English section)*)
- **Mise en place** : wS1; bP wS1-; wQ -wS1; bQ bP-; wA1 \\wQ; bG1 bQ-; wA1 \\bP
- **Attente** : move_illegal {'move': 'wA1 bP\\'}
- **Justification manuscrite** : « Le Pillbug ne peut pas déplacer la pièce qui vient d'être déplacée par l'autre joueur. » (feuille Pillbug, première exception). La fourmi des Blancs s'est déplacée vers la cellule au NO de bP au pli immédiatement précédent ; la tentative des Noirs d'utiliser la

capacité du Pillbug sur cette même fourmi — la lancer au SE de bP — doit être rejetée. (Le même lancer devient légal deux plis plus tard, ce qui constitue la mise en place du cas Co25.) Cas de garde du noyau étiqueté variante (protocole §2).

Co27 — One-Hive lie même la sauterelle : un point d’articulation ne peut pas sauter

- **Règle :** Règle One-Hive / Sauterelle (*Gen42 Hive rulesheet pp. 3, 6, 9*)
- **Mise en place :** wG1;bS1 wG1-;wQ -wG1;bQ bS1-
- **Attente :** moves_for_piece {'piece': 'wG1', 'moves': []}
- **Justification manuscrite :** La sauterelle est exemptée de la restriction de glissement (p. 10 : elle « peut sauter dans un espace ou hors d’un espace »), mais pas de One-Hive : « Si une pièce est la seule connexion entre deux parties de la Ruche, elle ne peut pas être déplacée. » (p. 3 NB). wG1 est entre wQ et la paire noire ; la soulever pour n’importe quel saut scinde la ruche en deux, donc malgré des lignes de saut dans les directions E et O, la sauterelle n’a aucun coup légal.

Co28 — Un scarabée ne peut pas être PLACÉ directement au sommet de la ruche

- **Règle :** Scarabée (NB de placement) (*Gen42 Hive rulesheet p. 5*)
- **Mise en place :** wS1;bS1 wS1-;wQ -wS1;bQ bS1-
- **Attente :** move_illegal {'move': 'wB1 wS1'}
- **Justification manuscrite :** « Lorsqu’il est placé pour la première fois, le Scarabée est placé de la même manière que toutes les autres pièces. Il ne peut pas être placé directement sur le dessus de la Ruche, même s’il peut y être déplacé plus tard. » (p. 5 NB). wB1 est encore en réserve ; la tentative de l’introduire au sommet de wS1 doit être rejetée. (Co13/Co14 vérifient que le même scarabée peut y grimper par un coup une fois placé.)

Co29 — L’araignée ne peut pas s’arrêter après un pas (« ni plus, ni moins »)

- **Règle :** Araignée (*Gen42 Hive rulesheet p. 7*)
- **Mise en place :** wG1;bS1 wG1-;wQ -wG1;bQ bS1-;wS1 \wQ;bG1 bQ-
- **Attente :** move_illegal {'move': 'wS1 \wG1'}
- **Justification manuscrite :** « L’Araignée se déplace de trois espaces par tour — ni plus, ni moins. » (p. 7). La cellule au NO de wG1 est à exactement un pas de glissement de la position de l’araignée, et aucun chemin légal de trois pas sans retour en arrière ne s’y termine (les deux marches de trois pas se terminent au NE de bS1 et au SE de wQ — cas Co08) ; un chemin passant par cette cellule la traverse au premier pas et ne peut pas s’y arrêter. Le coup d’un seul pas doit être rejeté.

Co30 — Reine entre deux pièces : deux glissements le long de l’épaule

- **Règle :** Reine / One-Hive / Liberté de mouvement (*Gen42 Hive rulesheet pp. 4, 9, 10*)
- **Mise en place :** wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bG1 bQ-
- **Attente :** moves_for_piece {'piece': 'wQ', 'moves': ['wQ -wB1', 'wQ /wS1']}
- **Justification manuscrite :** La reine touche wS1 (E) et wB1 (NE). La retirer garde la ruche entière (wB1 touche encore wS1), donc One-Hive autorise un coup. Glissements d’un espace (p. 4) : de ses quatre voisines vides, seules la cellule à l’O de wB1 (gardant le contact avec wB1) et la cellule au SO de wS1 (gardant le contact avec wS1) touchent encore la ruche après qu’elle se soulève ; les deux cellules plus à l’ouest ne touchent rien et sont exclues (p. 3 NB). Aucun des deux

glissements n'est derrière une porte (chacun est flanqué d'exactly une cellule occupée).
Attendu : exactement ces deux destinations.

A.2 Ensemble de vérification tactique (5 cas, H3)

Cas de correction de la recherche : mat en 1 par marche et par saut, pour les deux couleurs, et évitement d'auto-encerclement ; résolu 5/5 par la base MCTS à 400, 1600 et 6400 simulations.

Too1 — Les Blancs font mat en 1 : occuper la dernière liberté de la reine noire (SE de bQ)

- **Règle** : La fin de la partie (*Gen42 Hive rulesheet pp. 1, 11*)
- **Mise en place** : wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 -wQ;bG1 bQ-;wA1 -wG1;bG2 bQ/;wA2 -wA1;bB1 \bQ;wA3 -wA2;bA1 bS1\
- **Attente** : bestmove_to_cell {'target': 'bQ\\'}
- **Justification manuscrite** : La reine noire a exactement une voisine vide, la cellule au SE de bQ. Toute pièce blanche qui s'y pose achève l'encerclement et gagne immédiatement (« la partie se termine dès qu'une Reine est complètement encerclée par des pièces de n'importe quelle couleur », p. 11 ; mélange de couleurs permis, p. 1). La cellule est atteignable : une fourmi blanche peut parcourir le périmètre sud en un coup (l'entrée au-delà de bA1 n'est pas fermée par une porte), donc un coup gagnant existe. Aucun autre coup unique ne termine la partie. La recherche doit jouer sur cette cellule.

Too2 — Les Noirs font mat en 1 : occuper la dernière liberté de la reine blanche (SE de wQ)

- **Règle** : La fin de la partie (*Gen42 Hive rulesheet pp. 1, 11*)
- **Mise en place** : wS1;bS1 -wS1;wQ wS1-;bQ -bS1;wG1 wQ-;bG1 -bQ;wG2 wQ/;bG2 -bG1;wB1 \wQ;bA1 -bG2;wA1 wS1\;bA2 -bA1;wA2 wG1-
- **Attente** : bestmove_to_cell {'target': 'wQ\\'}
- **Justification manuscrite** : Miroir de Too1 avec les couleurs échangées et les Noirs au trait — cette paire est la vérification d'alternance des joueurs au niveau du coup : le motif gagnant doit être trouvé des deux côtés. La seule voisine vide de la reine blanche est la cellule au SE de wQ ; une fourmi noire l'atteint le long du périmètre sud (route via le SE de la colonne de bS1 : le pas d'entrée dans la cellule est flanqué par la cellule occupée wA1, donc le contact tient et aucune porte ne bloque). S'y poser achève l'encerclement : les Noirs gagnent (p. 11).

Too3 — Les Blancs font mat en 1 par saut de sauterelle par-dessus quatre pièces (E de bQ)

- **Règle** : Sauterelle / La fin de la partie (*Gen42 Hive rulesheet pp. 6, 11*)
- **Mise en place** : wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 -wQ;bG1 bQ/;wA1 \wQ;bB1 \bQ;wA2 \wA1;bA1 bS1\;wA3 \wA2;bA2 bQ\
- **Attente** : bestmove_to_cell {'target': 'bQ-'}
- **Justification manuscrite** : La seule voisine vide de la reine noire est la cellule à l'E de bQ. Plein est depuis wG1, au bout ouest, court la rangée occupée ininterrompue wQ, wS1, bS1, bQ ; le premier espace inoccupé le long de cette rangée est exactement la cellule gagnante, donc la sauterelle saute par-dessus quatre pièces et achève l'encerclement (p. 6 : « par-dessus un nombre quelconque de pièces ... jusqu'au premier espace inoccupé le long d'une rangée droite de pièces jointes » ; p. 11 : l'encerclement termine la partie). Les fourmis blanches peuvent aussi entrer en

marchant autour du périmètre — l'attendu est la cellule de destination, quelle que soit la pièce que la recherche envoie.

Too4 — Les Noirs font mat en 1 par saut de sauterelle par-dessus quatre pièces (O de wQ)

- **Règle** : Sauterelle / La fin de la partie (*Gen42 Hive rulesheet pp. 6, 11*)
- **Mise en place** : wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 \wQ;bG1 bQ-;wG2 \wS1;bA1 bQ\;wA1 wQ\;bA2 bA1\;wA2 /wQ;bA3 bA2\;wB1 \wG1
- **Attente** : `bestmove_to_cell {'target': '-wQ'}`
- **Justification manuscrite** : Miroir d'alternance de Too3 avec le saut noir. Les voisines de la reine blanche : E wS1, NO wG1 (placé au NO de wQ), NE wG2 (placé au NO de wS1 = NE de wQ), SE wA1, SO wA2 — cinq occupées, seule la cellule à l'O de wQ est vide. Plein est de ce trou court la rangée ininterrompue wQ, wS1, bS1, bQ avec bG1 au bout est (3,0) : depuis bG1, le premier espace inoccupé vers l'ouest le long de la rangée est exactement le trou, donc la sauterelle saute par-dessus quatre pièces et achève l'encerclement (pp. 6, 11). La queue sud des Noirs (bA1..bA3 au SE de bG1) garde les placements noirs antérieurs légaux et à l'écart des Blancs.

Too5 — Ne pas remplir la dernière liberté de sa propre reine

- **Règle** : La fin de la partie (*Gen42 Hive rulesheet p. 11*)
- **Mise en place** : wS1;bS1 -wS1;wQ wS1-;bQ -bS1;wG1 wQ-;bG1 -bQ;wG2 wQ/;bG2 -bG1;wB1 \wQ;bA1 -bG2;wA1 wS1\;bA2 -bA1
- **Attente** : `bestmove_avoid_cell {'target': 'wQ\\'}`
- **Justification manuscrite** : La cellule au SE de wQ est la dernière liberté de la reine blanche. Que les Blancs y placent ou y déplacent n'importe quelle pièce achève l'encerclement de leur propre reine — « la personne dont la Reine est encerclée perd la partie » (p. 11), quel que soit le camp qui a fourni la sixième pièce. Le placement est parfaitement légal (la cellule ne touche que des pièces blanches), donc seul le jugement de la recherche l'empêche. Tout coup sauf un coup atterrissant sur cette cellule convient ; la recherche ne doit pas y jouer. (Ceci n'affirme pas que les Blancs survivent à long terme — les Noirs menacent la même cellule — seulement que l'auto-encerclement immédiat est évité.)

Copie française (D-006) — version 1.0-draft 2026-10-09; les nombres et affirmations sont identiques au master anglais.

Annexe B — Architectures, décodeur et formats de données en détail

Sources : `python/hivenet/model.py`, `python/hivenet/graph_model.py`, `crates/hive-nn/` (font autorité) ; `docs/representations/` ; `docs/action-decoder.md`. Les comptes de paramètres sont mesurés (`count_params`), non estimés.

B.1 Bras grille — HiveNet (1.44 M paramètres)

Entrée : 77 plans $\times 32 \times 32$ (float32 dans $[0,1]$). Groupes de plans : 64 plans de pièces (camp propre/adverse $\times 8$ types d'insectes $\times$ niveau de pile 0–3+), sommets cloués One-Hive, cellule du dernier coup (étourdissement), régions de placement légales pour les deux camps, constante du camp au trait, scalaires de libertés des reines (les deux reines, /6), pli/100, bits de type de partie, comptes de réserves (/14).

Corps : tronc convolutif $3 \times 3 \rightarrow 8$ blocs résiduels de 96 canaux (deux conv 3×3 + BatchNorm chacun) ; les blocs 2 et 5 portent un biais de pooling global de style KataGo (canaux agrégés moyennelmax $\rightarrow$ linéaire $\rightarrow$ biais par canal). L'adjacence hexagonale en coordonnées axiales est un sous-ensemble de 7 cellules du voisinage 3×3 , de sorte que de simples convolutions 3×3 la couvrent ; les deux coins non voisins deviennent des poids morts apprenables.

Têtes : politique = convolution 1×1 vers 28 plans d'emplacements de pièces, aplatie en 28,672 logits spatiaux, plus un logit de passe issu des caractéristiques agrégées (`POLICY_SIZE` = 28,673) ; valeur = caractéristiques agrégées $\rightarrow 64 \rightarrow 3$ (victoire/nulle/défaite du point de vue du camp au trait).

B.2 Bras graphe — HiveGraphNet (1.47 M paramètres, +1.5%)

Entrée par position : jusqu'à 224 nœuds (cellules occupées + chaque cellule vide adjacente à la ruche — exactement l'univers de destinations du décodeur), chacun avec 56 caractéristiques : pour chaque niveau de pile 0–4, un bloc (présent, propriétaire-au-trait, type d'insecte en one-hot[8]) ; hauteur de pile/5 ; bits candidat-vide, cloué One-Hive, dernier coup, et les deux régions de placement. Un vecteur global de 23 valeurs (trait, pli/100, libertés des reines/6, réserves par type d'insecte/3 pour les deux camps, bits de type de partie) est concaténé à l'entrée de chaque nœud.

Corps : linéaire d'entrée vers 152 canaux $\rightarrow 8$ couches relationnelles à passage de messages : $h'_i = \text{ReLU}(W_{\text{self}} h_i + \sum_d W_d h_{\{n_i(d)\}} + b)$ avec six matrices de poids typées par direction (les six directions hexagonales comme types d'arêtes), résiduelles, masquées ; une couche sur trois ajoute un biais de pooling global masqué (moyennelmax $\rightarrow$ linéaire). La structure de voisinage est un tenseur d'indices (224×6) avec une ligne de remplissage à zéro — toutes les formes sont statiques, de sorte que l'export ONNX est à formes fixes et s'exécute sous le même chemin d'inférence Rust que le bras grille.

Têtes : valeur = pooling moyennelmax masqué || globaux $\rightarrow 64 \rightarrow 3$ (même convention). Politique = notation par candidat via le décodeur partagé : pour chaque (emplacement, destination) légal, le logit est MLP(plongement du nœud de destination || plongement de la source || plongement d'emplacements[32]), où le plongement de la source est le nœud où la pièce se tient pour les mouvements et un vecteur de réserve appris pour les placements ; un logit de passe appris. Les logits s'attachent aux coups, jamais aux positions de liste.

Variantes d'ablation (H-T3) : `untyped_edges` partage UNE SEULE matrice entre les six directions (0.54 M — les matrices typées sont le composant ablaté) ; `no_gpool` retire chaque biais de pooling (1.37 M).

B.3 Le décodeur d'actions partagé

Un coup est (emplacement de pièce relatif au camp 0–27, cellule de destination) plus la passe ; les emplacements 14–27 adressent les pièces adverses (lancers du Pillbug ; inertes en jeu de base). Le bras grille matérialise l'espace comme le tenseur plat à 28,673 sorties (emplacement × cellule du cadre) ; le bras graphe note les paires identiques par candidat. Les deux bras : masquage identique de l'ensemble légal, softmax sur exactement l'ensemble légal, cibles de distribution de visites MCTS identiques, départage déterministe vers l'indice plat le plus bas. La vérification 1 de la batterie pré-entraînement affirme une masse de probabilité nulle hors de l'ensemble légal via une vraie passe avant, pour le bras testé quel qu'il soit.

B.4 Format d'enregistrement v3 (818 octets)

Octets 0–83 : $28 \times (x, y, \text{niveau})$ en coordonnées du cadre, 255 = en main ; 84 camp au trait ; 85 identifiant du dernier coup (état d'étourdissement) ; 86 pli ; 87 bits de type de partie ; 88–91 libertés des reines et réserves (joueur au trait/adversaire) ; 92–95 masque de bits des clouages One-Hive ; 96–97 indice de politique du coup joué ; 98 issue du point de vue du joueur au trait — 0 défaite / 1 nulle / 2 victoire / **3 tronquée** (jamais une nulle) ; 99 version ; 100–107 estampille du modèle générateur (génération, hachage du réseau) ; 112–175 distribution de visites MCTS top-15 + total ; 176–177 compte de coups légaux ; 178–817 la liste des indices de politique légaux (plafond 320 ; facteur de branchement maximal mesuré 213) qui permet l'entraînement masqué sur l'ensemble légal dans les deux bras. Les constructeurs Rust et Python des deux encodages — plans et graphe — sont épinglés identiques à l'octet près par des tests dorés inter-langages exécutés chaque nuit (240 et 160 positions respectivement).

B.5 Recherche (partagée)

MCTS PUCT, $c = 1.4$, évaluation des feuilles par lots, valeurs terminales remontées exactement ; l'auto-jeu ajoute du bruit de Dirichlet à la racine ($\epsilon 0.25$), un échantillonnage en température sur les 12 premiers plis, la randomisation du plafond de simulations (25% des décisions à 128 simulations — enregistrées ; 75% à 32 — non enregistrées), l'abandon à -0.92 avec un audit sans abandon de 10% ; l'évaluation exécute 400 simulations, sans bruit, argmax déterministe (imposé par test, épinglé par config).

Copie française (D-006) — version 1.0-draft 2026-10-09; les nombres et affirmations sont identiques au master anglais.

Annexe C — Hyperparamètres, graines, commandes : fiche de reproduction complète

Chaque valeur ci-dessous est celle que les campagnes ont réellement exécutée (les fichiers `train-config.json` et `*-manifest.json` par exécution font autorité) ; rien ici n'est une recommandation.

C.1 Constantes expérimentales gelées

Constante	Valeur	Gelée par
Variante	jeu de base, règle d'ouverture de tournoi	D-007 / protocole v1.0
Plafond de coups	300 demi-coups ; troncature = issue à part entière	D-008/D-011 / protocole §3
Simulations d'auto-jeu (complètes/économiques)	128 / 32, fraction complète 0.25	protocole §5
Plis en température / abandon / audit	12 / -0.92 / 10%	matrice (pré-enregistrée)
Simulations d'évaluation / bruit / volume	400 / aucun / 100 parties/adversaire	D-019, D-027
Population d'adversaires	B-RND, B-HEU (poids sha256 do602f18...0b97a), B-MCTS@6400	D-017
Ouvertures	250 × 4 demi-coups, contenu sha256 63b318do...5af7b	D-025
T* (seuil à temps mural égal)	18.77 h (médiane des totaux grille s1-s3)	règle pré-enregistrée, calculé le 2026-09-16
Graines	1-5 par bras (4-5 ajoutées sous pré-engagement)	D-026 / D-031

C.2 Hyperparamètres d'entraînement (identiques pour les deux bras)

SGD, lr 0.02 en recuit cosinus jusqu'à lr/100 sur la durée de l'exécution, momentum 0.9, décroissance de poids 1e-4, lot 256, 2 époques par génération, poids de la perte de valeur 0.6, entropie croisée de politique masquée sur l'ensemble légal contre les distributions de visites MCTS, enregistrements tronqués exclus de la perte de valeur. Grille : 96 canaux × 8 blocs. Graphe : 152 canaux cachés × 8 couches, plongement d'emplacement 32. Aucune recherche d'hyperparamètres n'a été effectuée pour l'un ou l'autre bras (même budget de réglage nul, plan ch. 6) ; le seul changement d'optimiseur post hoc de toute l'étude est l'écrtage de gradient du supplément A1', explicitement à deux composants.

C.3 Dérivation des graines

Exécution d'entraînement (bras, graine) : $\text{base_seed} = 100,000 \times \text{graine}$; la génération g utilise $\text{base_seed} + g$ pour l'auto-jeu et l'entraînement ; le réseau gen-0 est une initialisation aléatoire à graine fixée exportée vers ONNX ($\text{torch.manual_seed} = \text{base_seed}$). Évaluation : graine du réseau 9000+gen (finales) / 9500 (ensembles T*), graines des adversaires 9101 (B-RND) et 9201 (B-MCTS), graines

d'arène telles que journalisées ; avec des ouvertures fixes, le calendrier est indépendant de la graine (prouvé par le test de hachage du calendrier).

C.4 Commandes

```
# one training run of the matrix (resumable per generation)
python3 scripts/run_comparison.py --arm grid --seed 1 --gens 10 --games 500
python3 scripts/run_comparison.py --arm graph --seed 1 --gens 10 --games 500
# ablations: --arm graph-untyped | graph-nogpool | graph-untyped-clip

# regenerate every table and figure from raw per-game records
python3 scripts/make_results.py
python/.venv/bin/python scripts/make_figures.py

# pre-training check battery on any shard set
bash scripts/run_h4_checks.sh '<abs>/gen000-*.bin' '<abs>/gen000-manifest.json'

# fresh-environment minimal reproduction (clone, build, replay, compare)
bash scripts/reproduce_minimal.sh /tmp/repro

# booklet (assembly -> HTML -> PDF)
python3 scripts/assemble_booklet.py && bash scripts/make_booklet_pdf.sh

# French/English numeric-identity check
python3 scripts/check_fr_numbers.py
```

C.5 Profil machine mesuré (tous les chiffres de temps mural)

Apple M1 Pro (10 cœurs, 16 Go, macOS 15.3.1) ; 4 threads de travail par exécution, exécutions séquentielles sous `caffeinate`. Inférence : grille CoreML 2.62 ms/eval, graphe CPU 3.67 ms/eval (CoreML plus lent pour le réseau graphe riche en opérations de collecte — mesuré, rapporté, imputé). Référence de débit d'entraînement sur MPS (lot 128, avant+arrière) : 274 (grille) / 138 (graphe) pos/s. Temps mural d'entraînement par exécution de 10 × 500 parties (génération en auto-jeu + entraînement, évaluation exclue) : grille 16.7–19.1 h, graphe 26.6–49.9 h ; évaluation ≈23–32 s/partie à 400 simulations.

C.6 Index des journaux et des décisions de cette étude

Protocole et gels : D-007/008/011/012/017/019/020/025. Campagnes : D-026 (principale), D-029 (ablations), D-030 (A1'), D-031 (extension à 5 graines avec pré-engagements). Journaux d'analyse : H6-2026-09-19-comparison-01 (3 graines), H6-2026-10-09-5seed-final-01 (finale), H7-2026-09-23 / 10-02 / 10-09 (A1, A2, A1'). Validation du moteur : l'ensemble H2-2026-09-09. Chaque cellule de table de ce livret est atteignable depuis l'un de ces éléments.

Annexe D — Pointeurs vers le dépôt

Everything not materialised in annexes A–C lives in the repository; each pointer names its identifiers and how to read them (plan ch. 23).

- **D.1 Configs, seeds, manifests.** `configs/` (matrix, baselines with sha256-pinned weights, pinned eval settings, ablation diffs); per-run `*-manifest.json` and `wallclock.json` under `data/runs/`.
- **D.2 Encoding conventions in full.** `docs/representations/{grid,graph, comparison-controls}.md`; `docs/action-decoder.md` — the normative prose behind annex B.
- **D.3 Claims register.** `paper/claims.md` — one row per claim: claim, evidence, section, limit. No row, no claim.
- **D.4 Experiment journal.** `journal/` — every run and measurement, negative results included; decision log in `state/decisions.md` (workspace repository).

Annexe E — Tableaux de résultats et figures (générés)

Inserés tels quels à l'assemblage depuis `results/` et `paper/figures/` — régénérables via `scripts/make_results.py` et `scripts/make_figures.py`.

H6 results — same-examples reading

Score = mean over non-truncated games (win 1 / draw 0.5 / loss 0); trunc = truncation rate; sens = truncations scored 0.5.

Arm	Seed	B-RND score / trunc	B-HEU score / trunc	B-MCTS score / trunc
grid	s1	0.995 / 0%	0.150 / 0%	0.125 / 0%
grid	s2	0.975 / 1%	0.080 / 0%	0.125 / 0%
grid	s3	0.990 / 1%	0.190 / 0%	0.075 / 0%
grid	s4	0.949 / 11%	0.105 / 0%	0.085 / 0%
grid	s5	0.995 / 0%	0.120 / 0%	0.210 / 0%
graph	s1	0.733 / 57%	0.025 / 0%	0.115 / 0%
graph	s2	0.981 / 20%	0.050 / 0%	0.125 / 0%
graph	s3	0.722 / 55%	0.090 / 0%	0.100 / 0%
graph	s4	0.688 / 44%	0.125 / 0%	0.120 / 0%
graph	s5	0.935 / 23%	0.035 / 0%	0.115 / 0%

Seed-level means (bootstrap 95%, unit = seed)

- grid vs B-RND: mean 0.981 [0.964, 0.994] (seeds: 0.995, 0.975, 0.990, 0.949, 0.995)
- grid vs B-HEU: mean 0.129 [0.098, 0.165] (seeds: 0.150, 0.080, 0.190, 0.105, 0.120)
- grid vs B-MCTS: mean 0.124 [0.087, 0.168] (seeds: 0.125, 0.125, 0.075, 0.085, 0.210)
- graph vs B-RND: mean 0.812 [0.710, 0.920] (seeds: 0.733, 0.981, 0.722, 0.688, 0.935)
- graph vs B-HEU: mean 0.065 [0.034, 0.100] (seeds: 0.025, 0.050, 0.090, 0.125, 0.035)
- graph vs B-MCTS: mean 0.115 [0.107, 0.121] (seeds: 0.115, 0.125, 0.100, 0.120, 0.115)

H6 results — same-wallclock reading

Score = mean over non-truncated games (win 1 / draw 0.5 / loss 0); trunc = truncation rate; sens = truncations scored 0.5.

Arm	Seed	B-RND score / trunc	B-HEU score / trunc	B-MCTS score / trunc
grid	s1	0.995 / 0%	0.150 / 0%	0.125 / 0%
grid	s2	0.975 / 1%	0.080 / 0%	0.125 / 0%
grid	s3	0.939 / 1%	0.145 / 0%	0.080 / 0%
grid	s4	0.949 / 11%	0.105 / 0%	0.085 / 0%
grid	s5	0.995 / 0%	0.120 / 0%	0.210 / 0%

Arm	Seed	B-RND score / trunc	B-HEU score / trunc	B-MCTS score / trunc
graph	s1	0.798 / 58%	0.045 / 0%	0.075 / 0%
graph	s2	0.926 / 39%	0.055 / 0%	0.110 / 0%
graph	s3	0.713 / 53%	0.060 / 0%	0.105 / 0%
graph	s4	0.631 / 39%	0.100 / 0%	0.150 / 0%
graph	s5	0.980 / 24%	0.045 / 0%	0.095 / 0%

Seed-level means (bootstrap 95%, unit = seed)

- grid vs B-RND: mean 0.971 [0.950, 0.991] (seeds: 0.995, 0.975, 0.939, 0.949, 0.995)
- grid vs B-HEU: mean 0.120 [0.098, 0.142] (seeds: 0.150, 0.080, 0.145, 0.105, 0.120)
- grid vs B-MCTS: mean 0.125 [0.090, 0.168] (seeds: 0.125, 0.125, 0.080, 0.085, 0.210)
- graph vs B-RND: mean 0.810 [0.697, 0.922] (seeds: 0.798, 0.926, 0.713, 0.631, 0.980)
- graph vs B-HEU: mean 0.061 [0.047, 0.081] (seeds: 0.045, 0.055, 0.060, 0.100, 0.045)
- graph vs B-MCTS: mean 0.107 [0.087, 0.130] (seeds: 0.075, 0.110, 0.105, 0.150, 0.095)

Arm contrast — same-examples

Graph – grid difference of seed-level means; bootstrap 95% over seeds (5 per arm, independent).

- vs B-RND: graph–grid = -0.169 [-0.272, -0.062]
- vs B-HEU: graph–grid = -0.064 [-0.111, -0.017]
- vs B-MCTS: graph–grid = -0.009 [-0.055, +0.028]

Arm contrast — same-wallclock

Graph – grid difference of seed-level means; bootstrap 95% over seeds (5 per arm, independent).

- vs B-RND: graph–grid = -0.161 [-0.278, -0.048]
- vs B-HEU: graph–grid = -0.059 [-0.087, -0.029]
- vs B-MCTS: graph–grid = -0.018 [-0.066, +0.025]

H7 ablation results (feeds booklet table H-T3)

Reference: H6 graph full method (journal H6-2026-09-19-comparison-01).

Ablation	Component removed	Outcome
A1 naive adjacency (graph–untyped)	direction-typed edge matrices → one shared matrix	Training diverged (NaN, generation 0, 3/3 seeds) — untrainable at parity settings; eval tables are artifacts of a NaN policy and are excluded as scores (journal H7-2026-09-23-a1-divergence-01). Attribution: typed edges contribute at minimum optimization stability.
A2 no global pooling (graph–nogpool)	global-pooling bias in all layers	NULL effect: diff vs full graph -0.001 [-0.170, +0.165] (B-RND), -0.007 [-0.043, +0.030] (B-HEU), -0.013 [-0.043, +0.013]

Ablation	Component removed	Outcome
		(B-MCTS); seed variance dominates; failure modes unchanged (journal H7-2026-10-02-a2-nogpool-01). Pooling is dispensable at this scale.

A1' supplement (graph-untyped-clip, two-component, D-030) | untyped edges + grad-clip 1.0 | **Trains finite; scores within the full arm's band** — diffs vs full graph: B-RND -0.066 $[-0.254, +0.130]$, B-HEU $+0.032$ $[-0.011, +0.078]$, B-MCTS $+0.008$ $[-0.040, +0.063]$. NEVER attributed to typing alone (clip confound). With A1: typed edges' measurable contribution at this scale is concentrated in optimization stability (journal H7-2026-10-09-a1prime-01). |

Score / cost table (H6 task 7; fig2)

Metric	Grid arm	Graph arm
Parameters	1.44 M	1.47 M (+1.5%)
Best-provider inference (b1)	2.62 ms (CoreML)	3.67 ms (CPU)
Mean run wall-clock (10 gens $\times$ 500 games)	18.0 h	36.7 h (2.0 $\times$)
Mean self-play cost	13.0 s/game	26.4 s/game
Training throughput benchmark (MPS, batch 128, fwd+bwd)	274 pos/s	138 pos/s
Population score, same-examples	0.411	0.331
Population score, same-wall-clock (T*=18.77 h)	0.405	0.326

Population score = mean over the three frozen opponents of the seed-mean score (truncations excluded, reported separately in the results tables). Wall-clock = self-play generation + training per run (evaluation games excluded), mean over the five seeds; self-play cost = that wall-clock / 5,000 games. Sources: results-*.csv, wallclock.json per run, comparison-controls.md measurements (journal H5-2026-09-10-encoders-01); journal H6-2026-09-19-comparison-01.

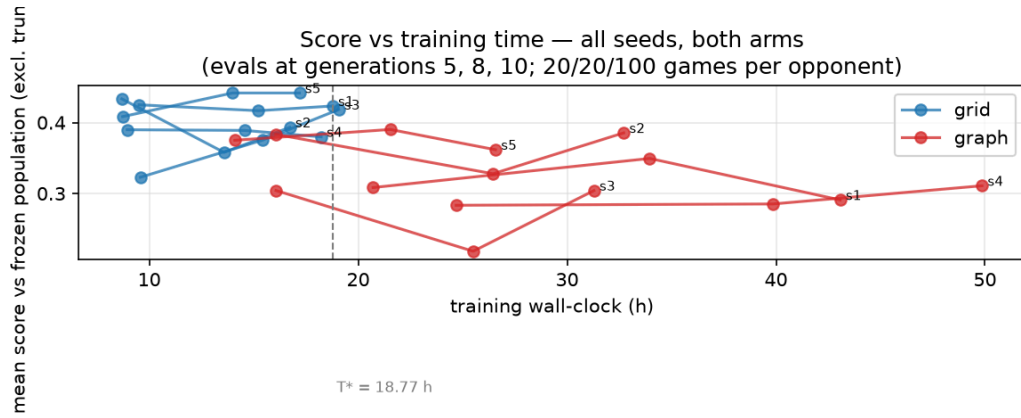


Fig. 1 — Score moyen contre la population gelée vs temps mural d'entraînement, toutes graines, les deux bras (évaluations aux générations 5, 8, 10 ; pointillé = T^*).

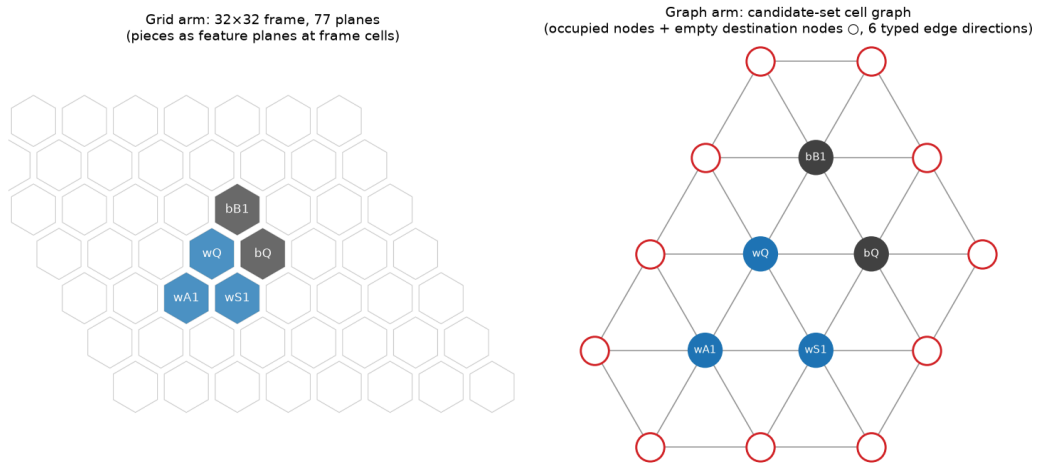


Fig. 3 — Les deux encodages d'une même position : plans en grille dans le cadre 32x32 (gauche) et graphe de cellules avec ses six relations typées par direction (droite).

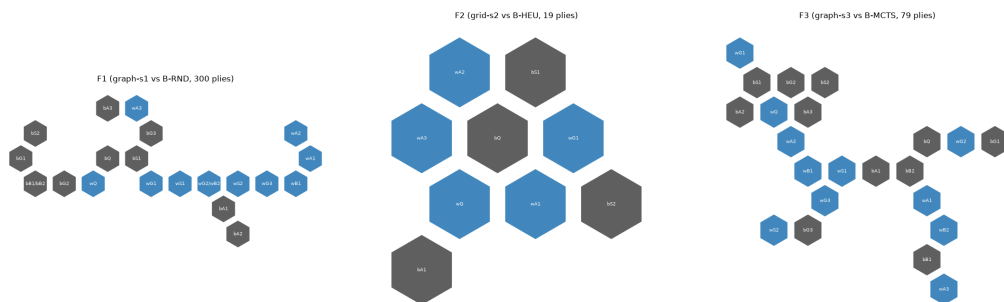


Fig. 4 — Trois positions d'échec commentées F1–F3 (détails et statut de reproduction ci-dessous).

Three commented failure positions (H6 task 7)

Selection criteria are mechanical, stated in `scripts/make_figures.py::fig4`, and applied over the raw per-game CSV records; each selected game is reproduced deterministically (fresh per-game engine seeds) and verified against its CSV row.

F1: graph vs B-RND — longest truncated game (a won position it cannot close: wins material, then shuffles to the 300-ply cap)

- game: graph-s1 vs B-RND, opening 2, A white, 300 plies
- reproduction verified against CSV row: YES
- GameString: Base;InProgress;White[151];wG1;bS1 \wG1;wQ /wG1;bQ -bS1;wS1 wG1-;bG1 -bQ;wG2 wS1-;bB1 bQ;/wS2 wG2-;bA1 -bG1;wG3 wS2-;bS2 \bA1;wA1 wG3\;bA2 bB1-;wB1 wA1\;bB1 bS1;wB2 wB1-;bB1 wG1;wA2 wB2\;bA2 bS2-;wA3 wB2;/bG2 \bS2;wA2 wA3;/bB2 bG2;/wA2 -bG2;bA1 /bQ;wA3 -wA2;bG3 bS1;/wA3 wB2\;bA3 \bG3;wA3 wB2;/bA1 /wS1;wA3 \bA3;bA1 wA3-;wB2 wA1-;bA1 wS2;/wB1 /wA1;bA1 bB2-;wA2 wA3;/bA1 \bB2;wA2 -bA1;bB1 bS1;wA2 wG3;/bA1 bB2\;wB1 wA1;bA1 /bG1;wB2 wB1;bA1 wG2;/wB2 wB1\;bA1 bG3-;wB2 /wB1;bA1 bG3\;wB2 -wB1;bB2 \bG2;wB...

F2: grid vs B-HEU — shortest decided loss (the heuristic's queen-targeting tactics strike before the net consolidates)

- game: grid-s2 vs B-HEU, opening 2, A black, 19 plies
- reproduction verified against CSV row: YES
- GameString: Base;WhiteWins;Black[10];wG1;bS1 \wG1;wQ /wG1;bQ -bS1;wA1 wG1-;bA1 bS1;/wA1 \bA1;bQ -wG1;wQ /bQ;bS2 -bS1;wA2 \wA1;bS2 /wQ;wA2 -bS1;bS2 wG1\;wA1 -bS2;bA1 /wA1;wA3 \wA2;bA1 /wQ;wA3 -bQ

F3: graph vs B-MCTS — longest drawn game (avoids losing without ever generating winning threats)

- game: graph-s3 vs B-MCTS, opening 19, A white, 79 plies
- reproduction verified against CSV row: YES
- GameString: Base;Draw;Black[40];wS1;bA1 wS1-;wA1 \wS1;bB1 bA1-;wA2 -wA1;bA2 /bB1;wQ \wA2;bQ bB1-;wQ wA2;/bA2 /wA2;wQ \wA2;bA2 -wQ;wB1 -wS1;bA3 bA1;/wA1 bQ-;bA3 bA2;/wA1 /bA1;bG1 bA1;/wA1 bQ-;bG2 -bA3;wA1 -bG2;bG3 /bB1;wA1 /bQ;bB1 wA1;wB2 /wS1;bG2 bA3-;wB2 wS1;bB2 -bQ;wB2 /wS1;bA3 bG2\;wB2 -bG3;bQ bB1-;wB2 /bG3;bS1 bG2-;wB2 /bB1;bS1 -bG2;wB2 /bQ;bQ bB2-;wB2 /bB1;bS2 bG2-;wB2 bB1\;bB1 wB2;wG1 /wS1;bQ bG1-;wG1 \bS1;bS2 wG1;/wA3 -wG1;bS2 -wA3;wG2 /wS1;bG2 -wA2;wG2 -bG1;bG2 bS1-;wG2 bQ-;bG1 wG2-;wG3 /wS1;bG3 wS1...

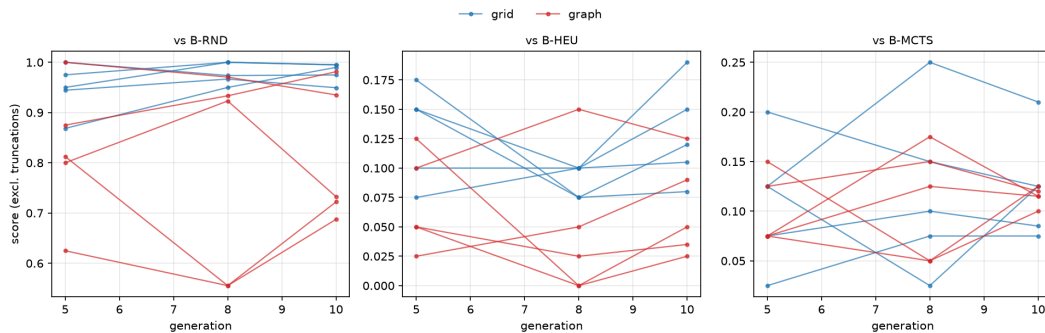


Fig. 5 — Trajectoires de score par adversaire (moyenne sur 5 graines, bande min-max ; points de contrôle finaux par génération ; 100 parties/adversaire à 400 simulations).

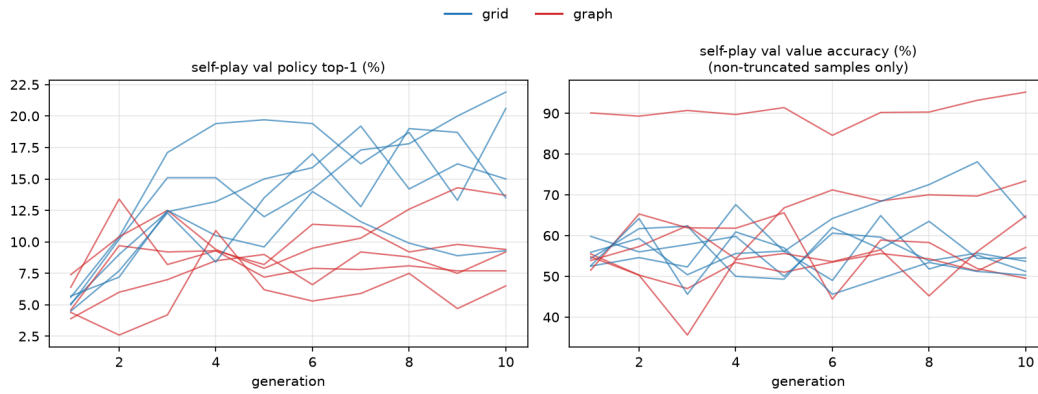


Fig. 6 — Métriques d’entraînement par génération (top-1 politique et exactitude valeur à l’époque 1, 10 exécutions de la campagne principale ; exécutions d’ablation exclues).

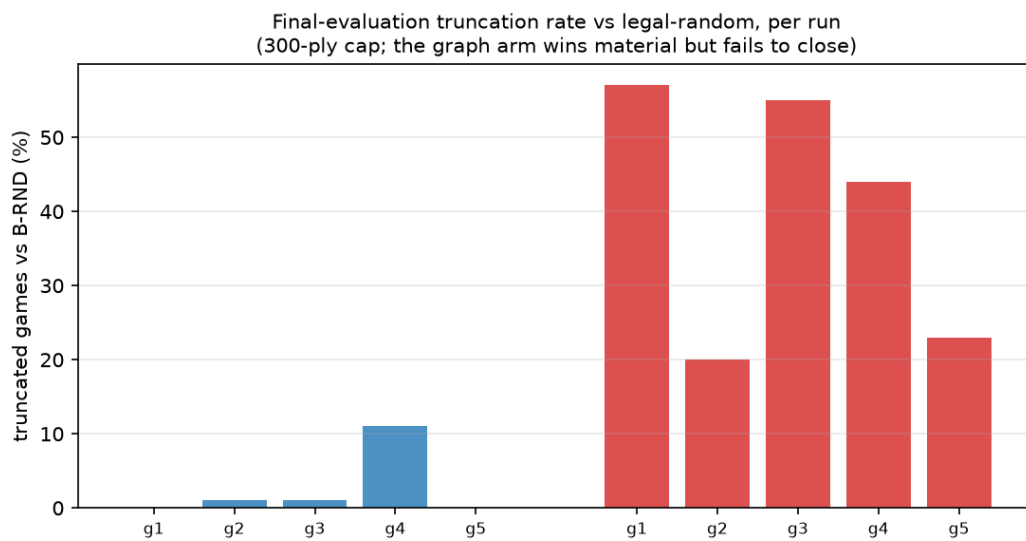


Fig. 7 — Taux de troncature en évaluation finale contre l’aléatoire légal, par exécution (points de contrôle finaux, 5 graines par bras ; troncature rapportée séparément des nulles, invariant 7).